

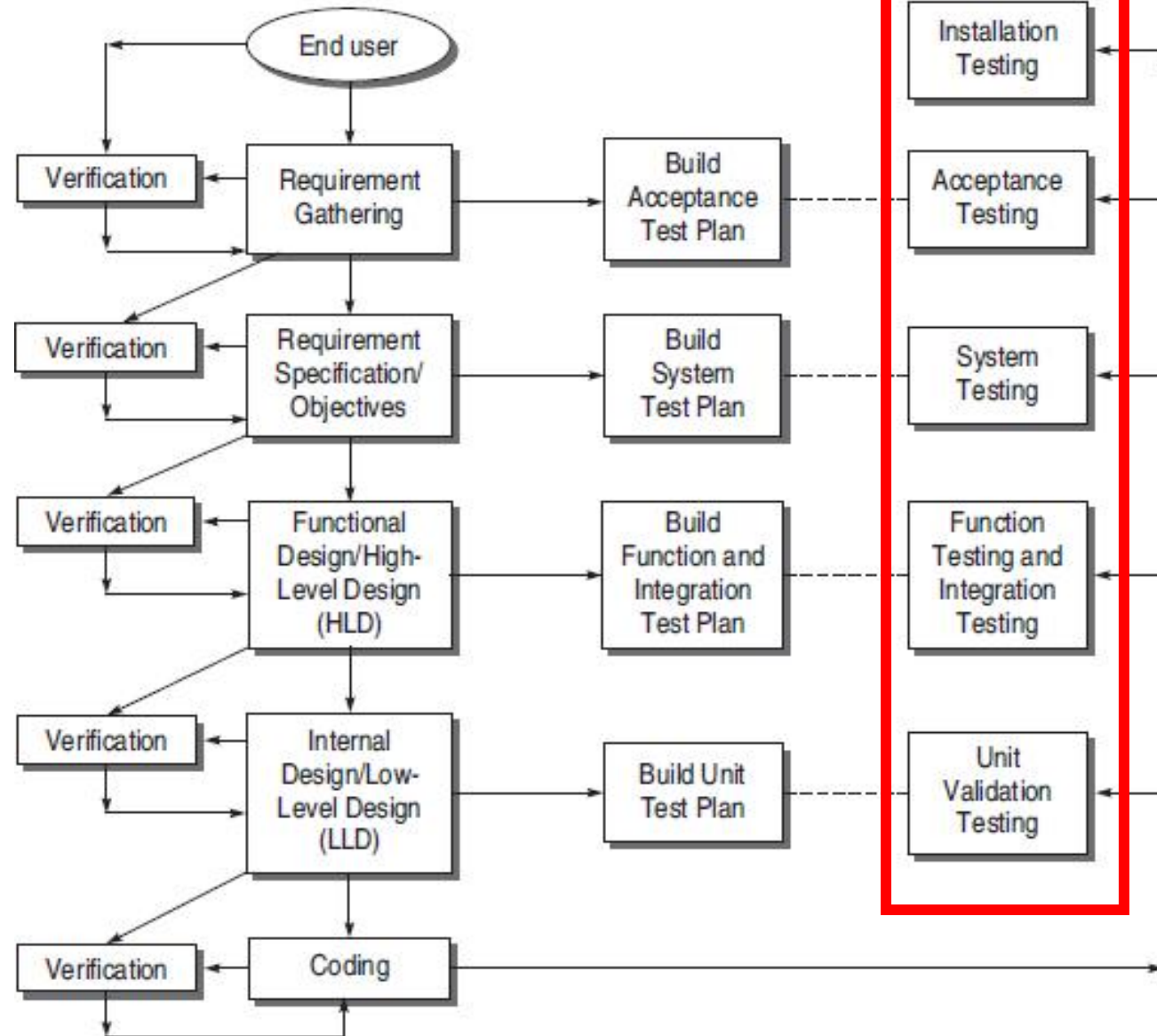


CSE 4513

Lec – 12 (Part 2)



VALIDATION ACTIVITIES



INTEGRATION TESTING



- ✓ In **integration testing** smaller units are integrated into larger units and larger units into the overall system.
- ✓ This differs from unit testing in that units are no longer tested independently but in groups, the focus shifting from the individual units to the interaction between them.
- ✓ The purpose of integration testing is to verify the **functional**, **performance**, and **reliability** between the modules that are integrated.

INTEGRATION TESTING



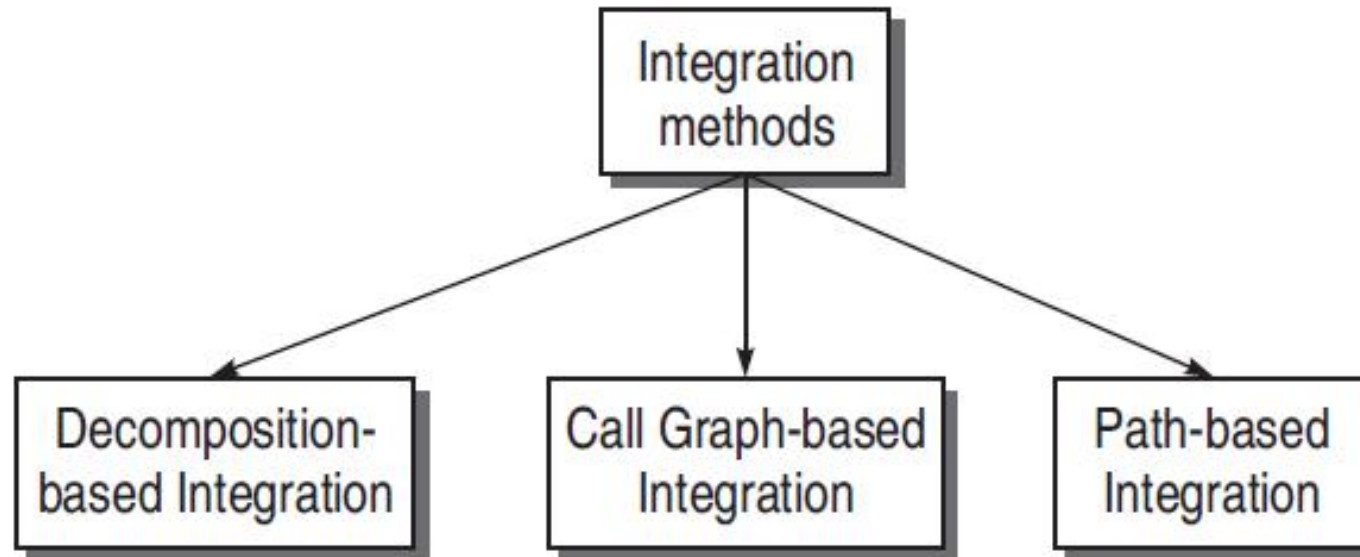
Why do we do integration testing?

- ✓ Unit tests only test the unit in isolation
- ✓ Many failures result from faults in the interaction of subsystems
- ✓ Often many **Off-the-shelf components** are used that cannot be unit tested
- ✓ Without integration testing the system test will be very time-consuming
- ✓ Failures not discovered in integration testing will be found after the system is deployed and can be very expensive to fix.

INTEGRATION TESTING



There are three approaches for integration testing.

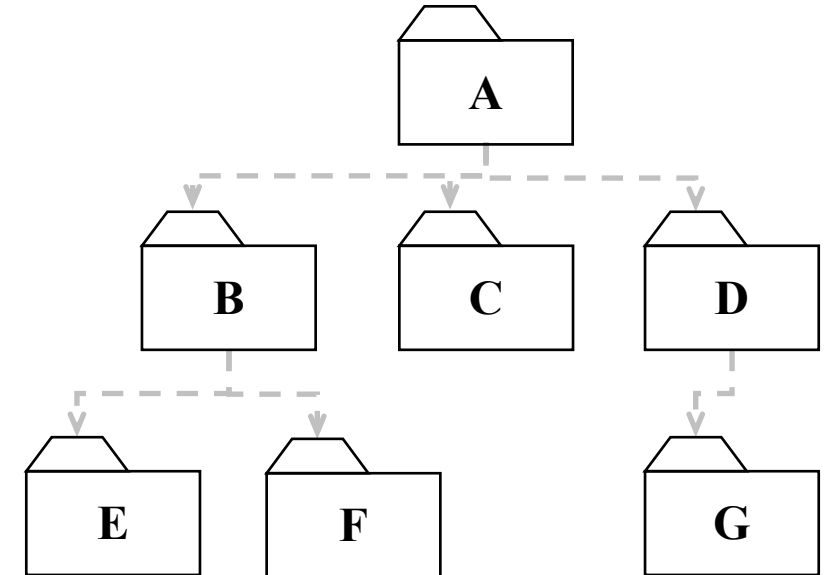
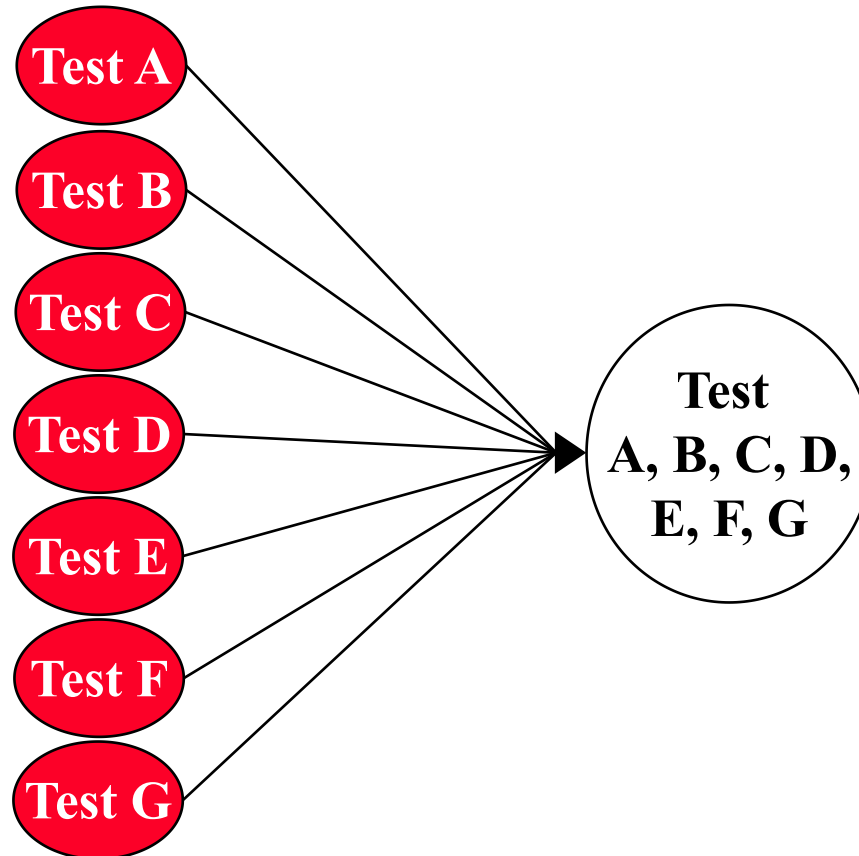


INTEGRATION TESTING : DECOMPOSITION BASED



Big-Bang Approach

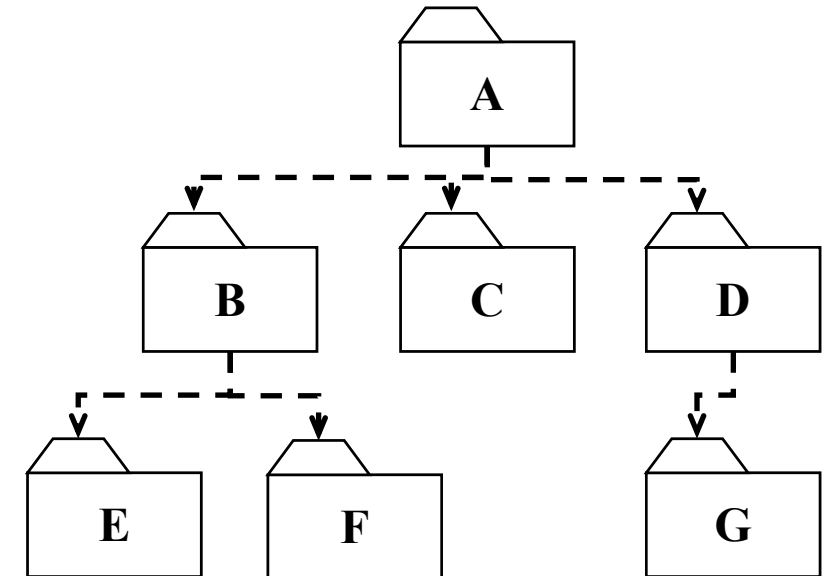
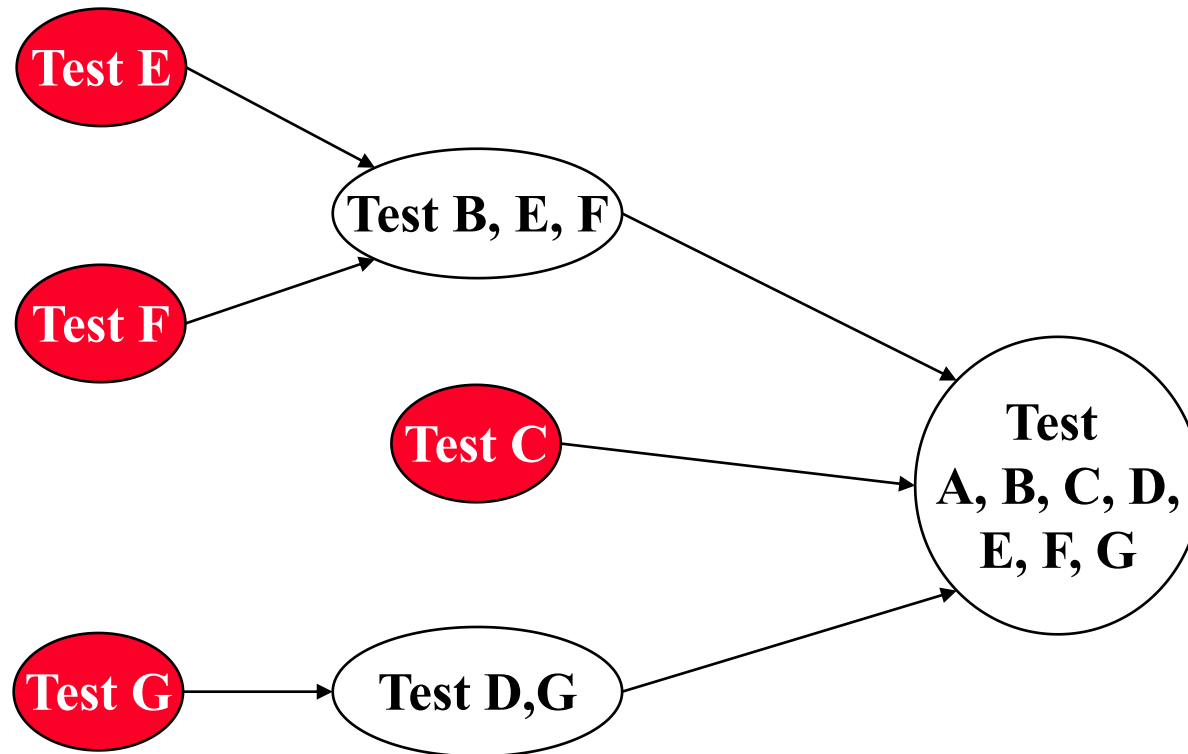
- ✓ Unit test each of the subsystems, and then do one massive integration test, in which all the subsystems are immediately tested together.
- ✓ Don't try this!! Why: The interfaces of each of the subsystems have not been tested yet.



INTEGRATION TESTING : DECOMPOSITION BASED

Bottom-up Testing Strategy

- ✓ The subsystems in the lowest layer of the call hierarchy are tested individually
- ✓ Then the next subsystems are tested that call the previously tested subsystems
- ✓ This is repeated until all subsystems are included
- ✓ Drivers are needed.

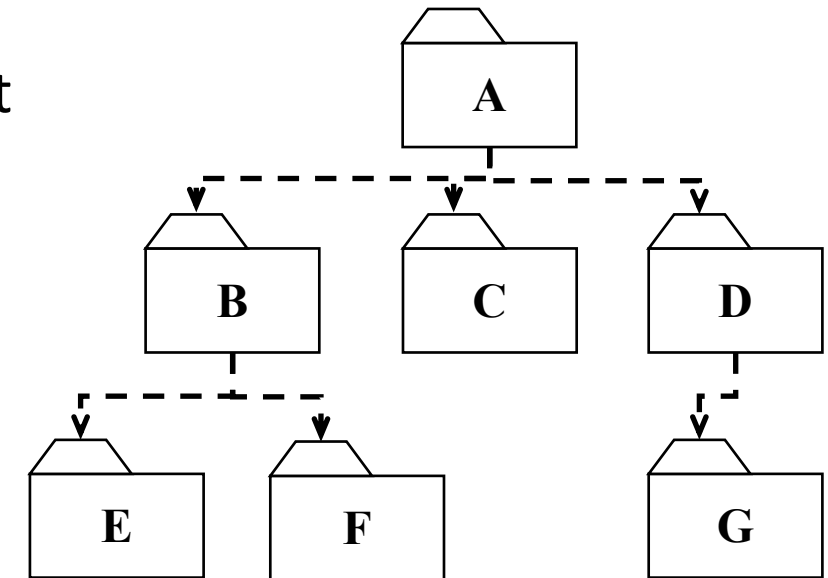
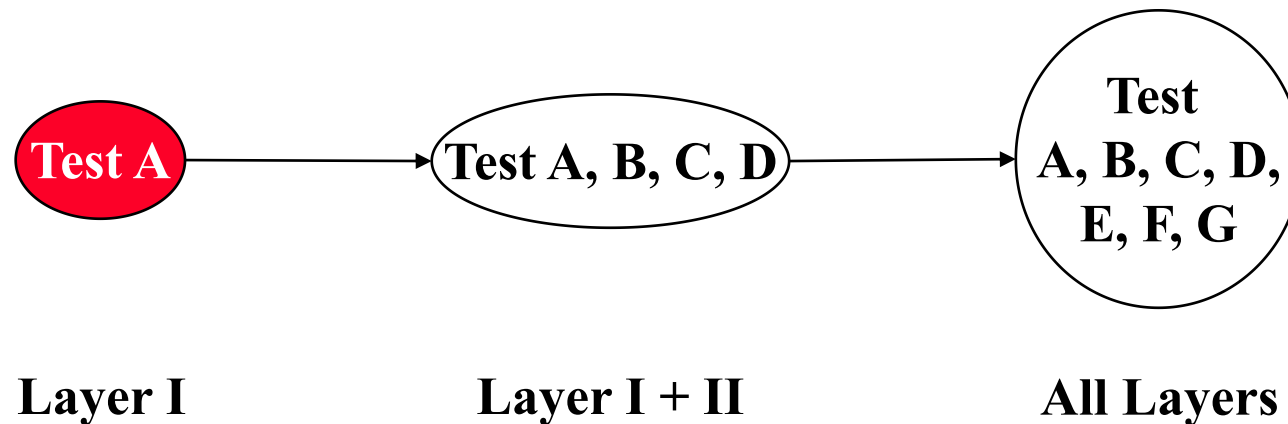


INTEGRATION TESTING : DECOMPOSITION BASED



Top-Down Testing Strategy

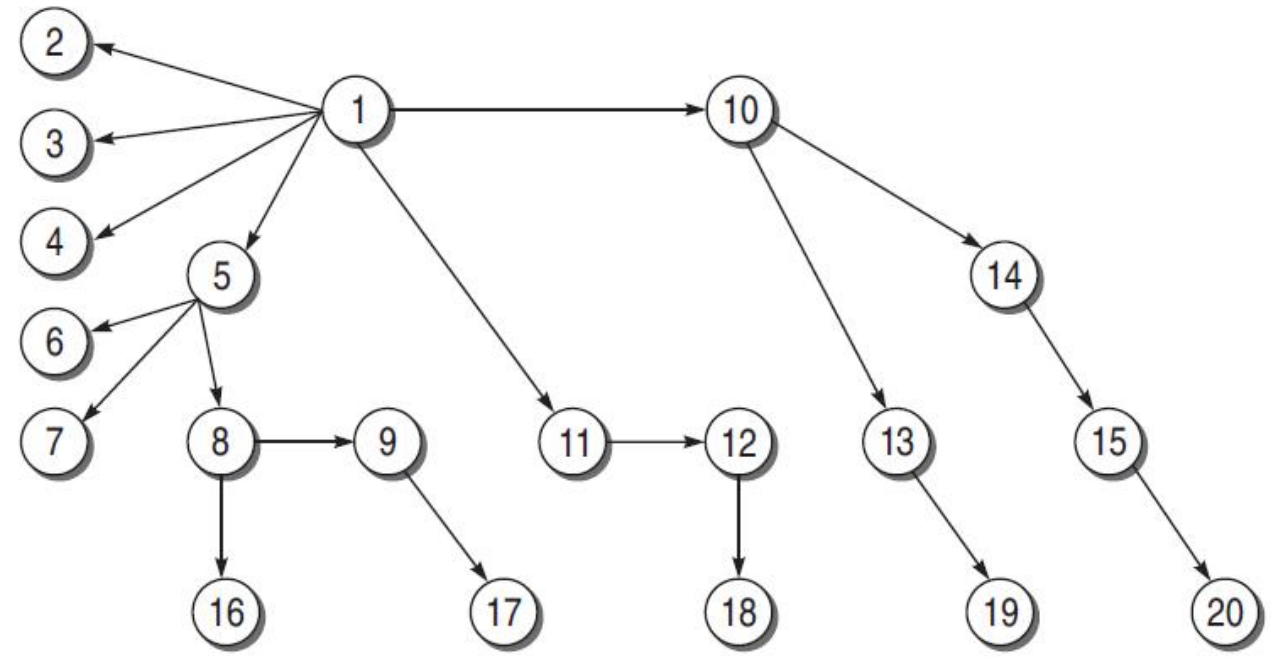
- ✓ Test the top layer or the controlling subsystem first
- ✓ Then combine all the subsystems that are called by the tested subsystems and test the resulting collection of subsystems
- ✓ Do this until all subsystems are incorporated into the test
- ✓ Stubs are needed to do the testing.



INTEGRATION TESTING : CALL GRAPH BASED



- ✓ A call graph is a directed graph, wherein the nodes are either modules or units, and a directed edge from one node to another means one module has called another module.
- ✓ The call graph can be captured in a matrix form which is known as the adjacency matrix.
- ✓ The idea behind using a call graph for integration testing is to avoid the efforts made in developing the **stubs** and **drivers**.



Call Graph

[illegible]

INTEGRATION TESTING: CALL GRAPH BASED



There are two types of integration testing based on call graph:

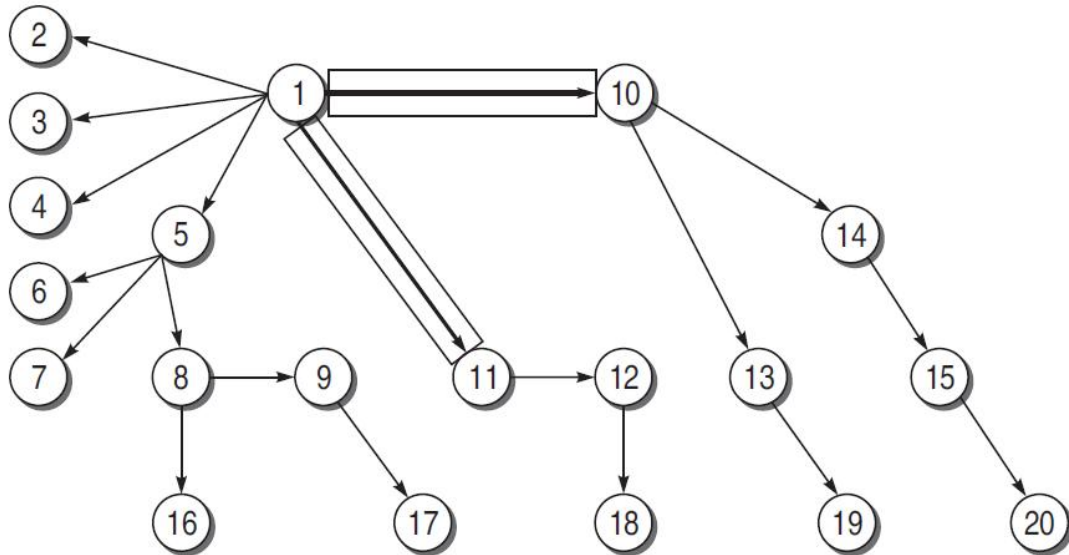
- ✓ Pair-wise Integration
- ✓ Neighborhood Integration

Pair-wise Integration

A form of integration testing that targets pairs of components that work together, as shown in a call graph.

If we consider only one pair of calling and called modules, then we can make a set of pairs for all such modules, as shown in Fig., for pairs 1–10 and 1–11. The resulting set will be the total test sessions which will be equal to the sum of all edges in the call graph.

For example, in the call graph shown in Fig., the number of test sessions is 19 which is equal to the number of edges in the call graph.

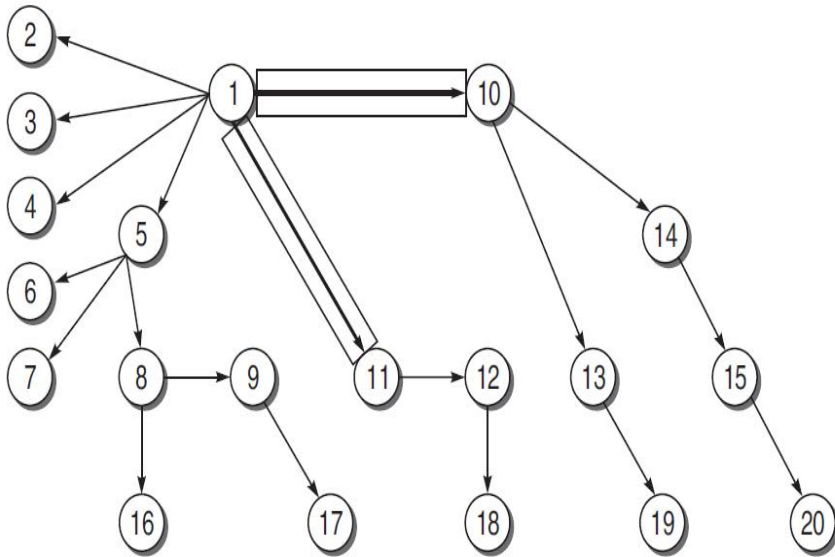


INTEGRATION TESTING: CALL GRAPH BASED



There are two types of integration testing based on call graph:

- ✓ Pair-wise Integration
- ✓ Neighborhood Integration



Node	Neighbourhoods	
	Predecessors	Successors
1	-----	2,3,4,5,10,11
5	1	6,7,8
8	5	9,16
9	8	17
10	1	13,14
11	1	12
12	11	18
13	10	19
14	10	15
15	14	20

sessions in
ed
graph, then
isor as well as

The total test sessions in neighborhood integration can be calculated as:

$$\begin{aligned}
 \text{Neighborhoods} &= \text{nodes} - \text{sink nodes} \\
 &= 20 - 10 \\
 &= 10
 \end{aligned}$$

where **sink node** is an instruction in a module at which the execution terminates.

INTEGRATION TESTING: PATH BASED



Source node It is an instruction in the module at which the execution **starts** or **resumes**. The nodes where the **control is being transferred after calling** the module are also **source nodes**.

Sink node It is an instruction in a module at which the **execution terminates**. The nodes from which the **control is transferred** are also sink nodes.

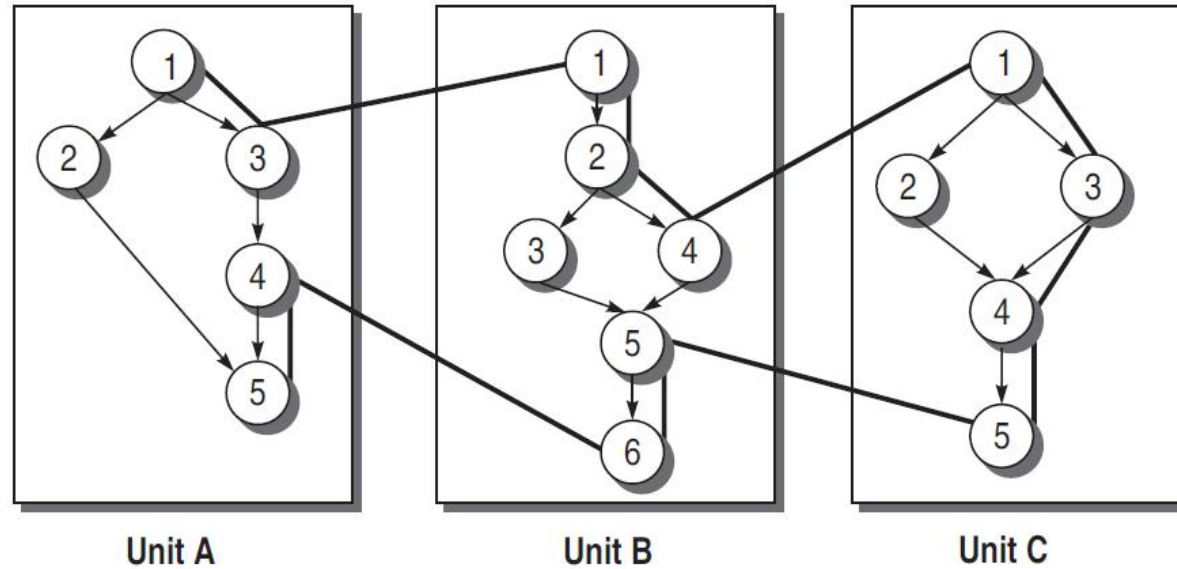
Module execution path (MEP) It is a sequence of statements that begins with a source node and ends with a sink node, with no intervening sink nodes.

message is a programming language mechanism by which one unit transfers control to another unit, and acquires a response from the other unit.

MM-Path It is a path consisting of MEPs and messages. The path shows the sequence of executable statements.

MM-path graph It can be defined as an extended flow graph where nodes are MEPs and edges are messages.

EXAMPLE

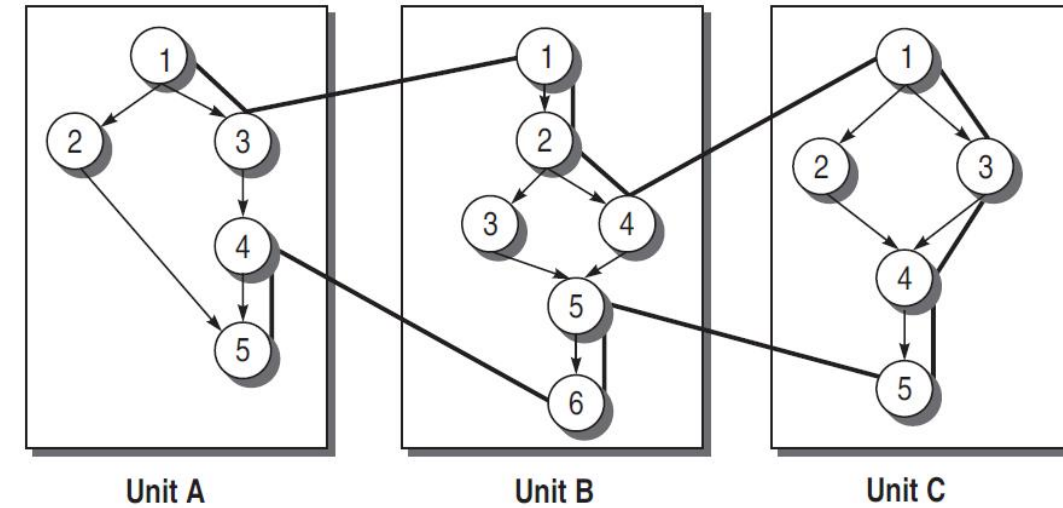
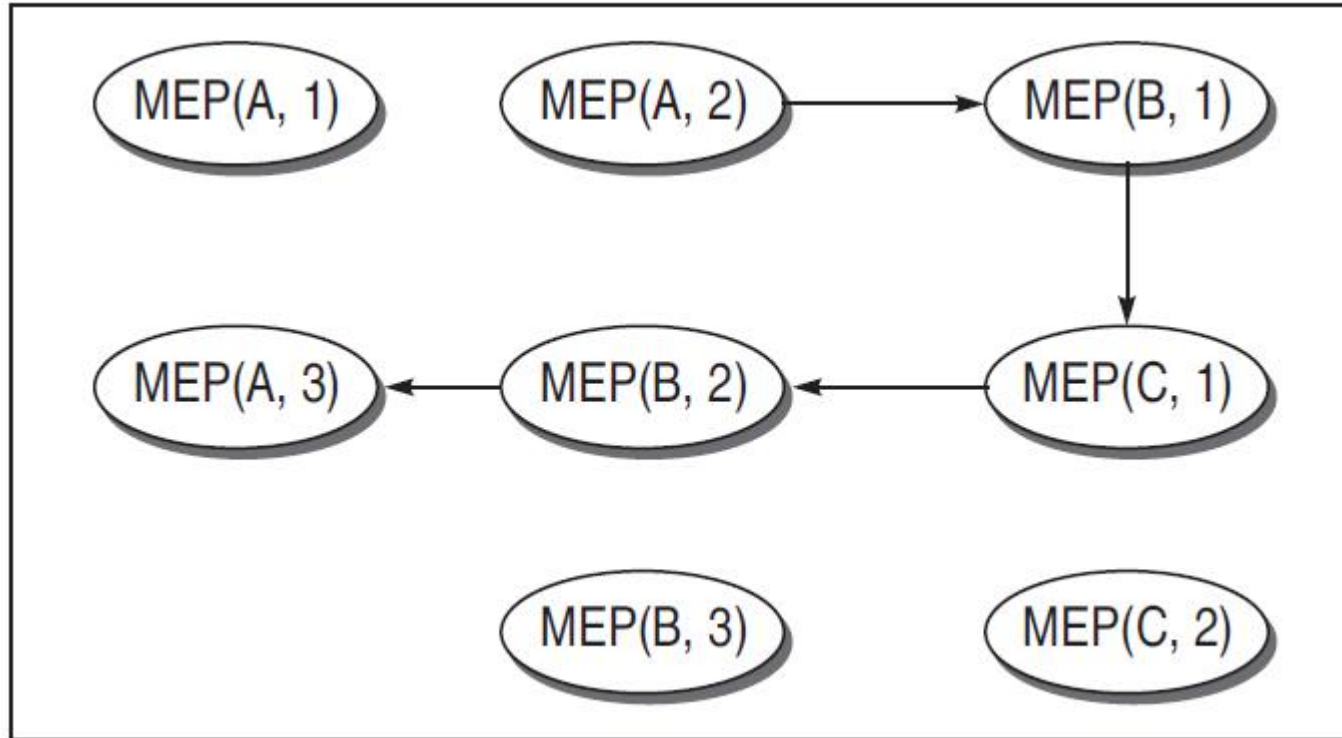


	Source Nodes	Sink Nodes	MEPs
Unit A	1,4	3,5	$MEP(A,1) = \langle 1,2,5 \rangle$ $MEP(A,2) = \langle 1,3 \rangle$ $MEP(A,3) = \langle 4,5 \rangle$
Unit B	1,5	4,6	$MEP(B,1) = \langle 1,2,4 \rangle$ $MEP(B,2) = \langle 5,6 \rangle$ $MEP(B,3) = \langle 1,2,3,4,5,6 \rangle$
Unit C	1	5	$MEP(C,1) = \langle 1,3,4,5 \rangle$ $MEP(C,2) = \langle 1,2,4,5 \rangle$

EXAMPLE



The MM-path graph for this example is

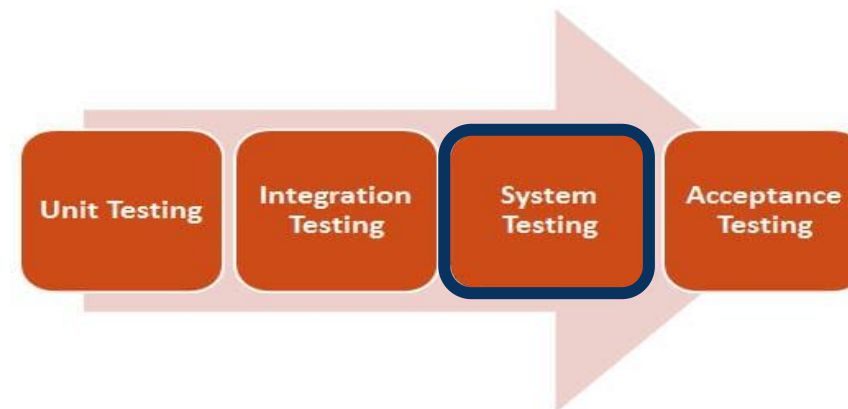


MEPs
MEP(A,1) = <1,2,5>
MEP(A,2) = <1,3>
MEP(A,3) = <4,5>
MEP(B,1) = <1,2,4>
MEP(B,2) = <5,6>
MEP(B,3) = <1,2,3,4,5,6>
MEP(C,1) = <1,3,4,5>
MEP(C,2) = <1,2,4,5>

SYSTEM TESTING



- ✓ is a series of different tests to test the whole system on various grounds (functional & non functional) where bugs have the probability to occur. The ground can be performance, security, maximum load, etc.
- ✓ **SYSTEM TESTING** is a level of testing that validates the complete and fully integrated software product.
- ✓ The purpose of a system test is to evaluate the end-to-end system specifications.
- ✓ System Testing (ST) is a **black box** testing technique performed to evaluate the complete system.
- ✓ is usually carried out by a team that is independent of the development team in order to measure the quality of the system unbiased.

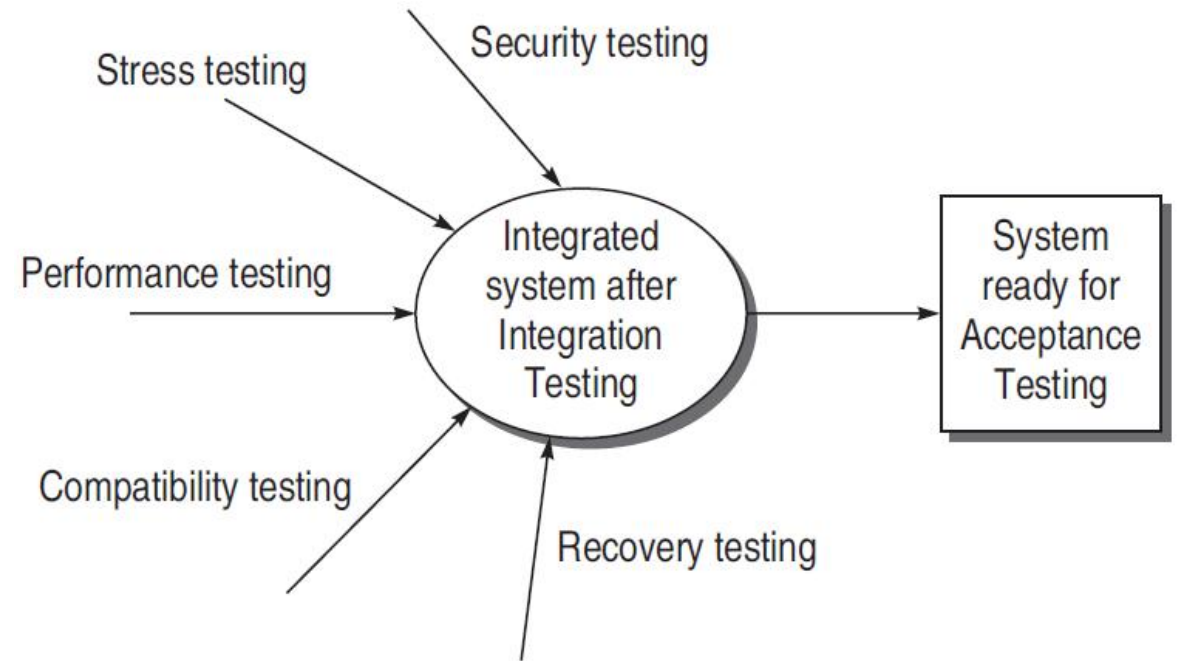


SYSTEM TESTING



System testing includes:

- ✓ Functional Testing
 - Validates functional requirements
- ✓ Performance Testing
 - Validates non-functional requirements



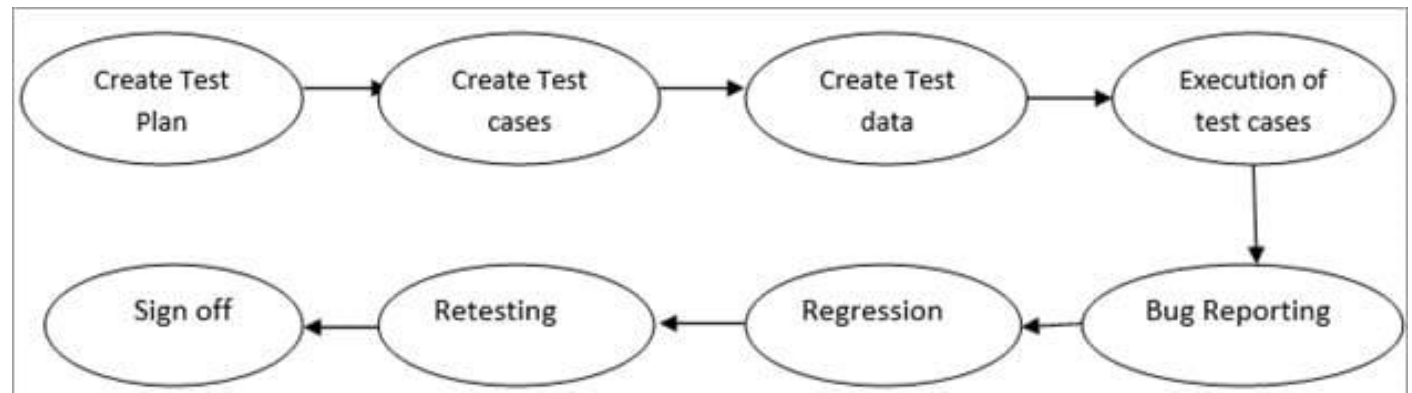
When we are system testing, we are testing all subsystems together

SYSTEM TESTING



Steps in system Testing

1. The very first step is to create a Test Plan.
2. Create System Test Cases and test scripts.
3. Prepare the test data required for this testing.
4. Execute the system test cases and script.
5. Report the bugs. Re-testing the bugs once fixed.
6. **Regression** testing to verify the impact of the change in the code.
7. Repetition of the testing cycle until the system is ready to be deployed.
8. Sign off from the testing team.



SYSTEM TESTING



Procedure To Write System Test Cases

- ✓ System test cases cover all the scenarios & use cases
- ✓ it covers functional, non- functional, user interface, security-related test cases.

System test cases commonly include the below fields:

Test Case ID	Test Suite Name	Description	Steps	Test Data	Expected Result	Actual Result	Pass/Fail	Remarks
TC_01	ABC							
TC_02								

ACCEPTANCE TESTING

USER ACCEPTANCE TESTING(UAT)



Testing is performed by the Client of the application to determine whether the application is developed as per the requirements specified by him/her.



Acceptance testing is the formal testing conducted to determine whether a software system satisfies its acceptance criteria and to enable clients to determine whether to accept the system or not.

USER ACCEPTANCE TESTING(UAT)



Need of UAT:

✓ Developers code software based on requirements document which is their "own" understanding of the requirements and **may not actually be what the client needs from the software.**

1

- Developers have included features on their "own" understanding

✓ Requirements changes during the course of the project may not be communicated effectively to the developers.

2

- Requirements changes "not communicated" effectively to the developers

Running acceptance tests ensures that no requirement change has happened in the meantime and that everything is as it should be to satisfy the customer.

USER ACCEPTANCE TESTING (UAT)



Entry Criteria

- ✓ System testing is complete and defects identified are either fixed or documented.
- ✓ Acceptance plan is prepared and resources have been identified.
- ✓ Test environment for the acceptance testing is available.

Exit Criteria

- Acceptance decision is made for the software.
- In case of any warning, the development team is notified.

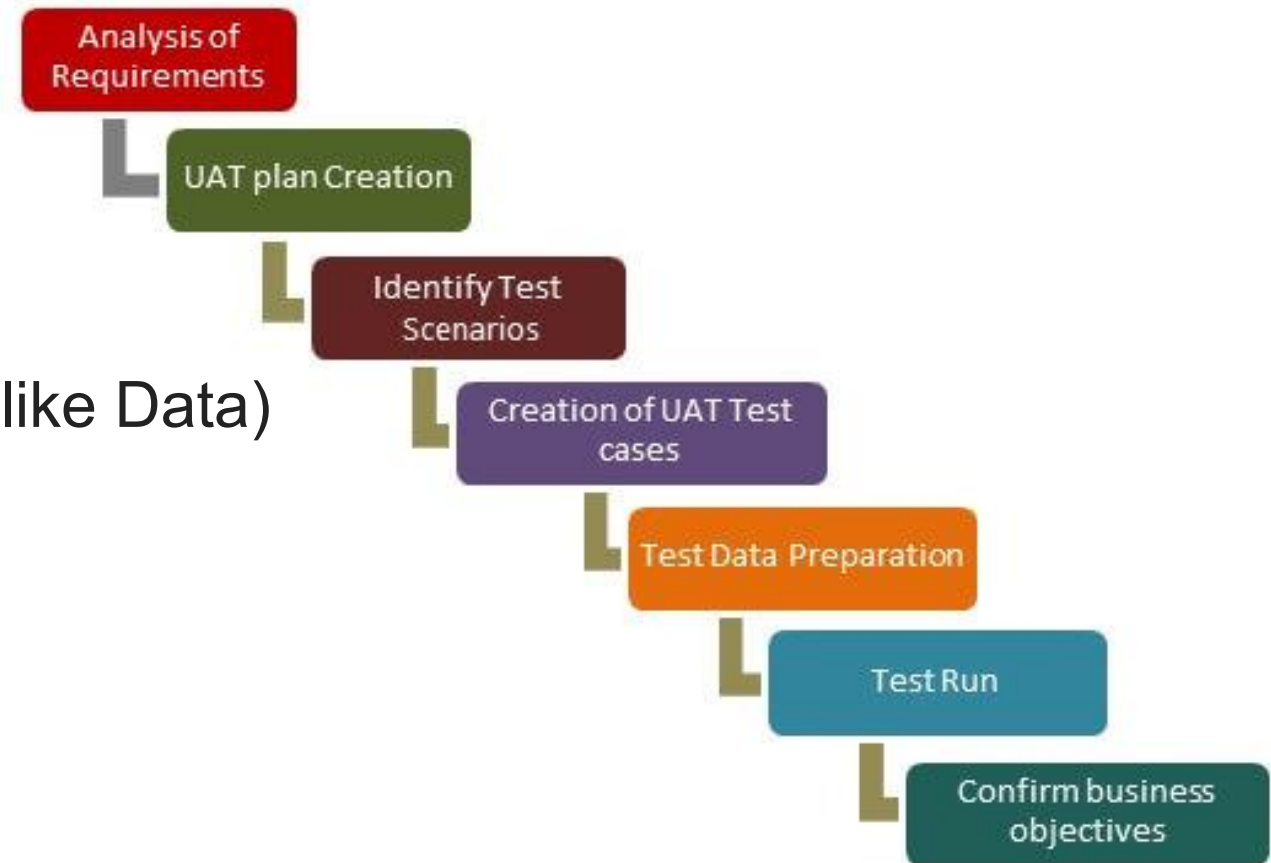
USER ACCEPTANCE TESTING (UAT)



UAT Process:

Once Entry criteria for UAT are satisfied, following are the tasks need to be performed by the testers:

- ✓ Analysis of Business Requirements
- ✓ Creation of UAT test plan
- ✓ Identify Test Scenarios
- ✓ Create UAT Test Cases
- ✓ Preparation of Test Data(Production like Data)
- ✓ Run the Test cases
- ✓ Record the Results
- ✓ Confirm business objectives



USER ACCEPTANCE TESTING (UAT)



Types of Acceptance Testing

Acceptance testing is classified into the following two categories:

- ✓ **Alpha Testing** Tests are conducted at the development site by the end users. The test environment can be controlled a little in this case.
- ✓ **Beta Testing** Tests are conducted at the customer site and the development team does not have any control over the test environment.

ALPHA TESTING



Alpha testing is one of the most common software testing strategy used in software development. Its specially used by product development organizations.

- ✓ This test takes place at the developer's site. Developers observe the users and note problems.
- ✓ Alpha testing is testing of an application when development is about to complete.
- ✓ Minor design changes can still be made as a result of alpha testing.
- ✓ Alpha testing is typically performed by a group that is independent of the design team, but still within the company, e.g. in-house software test engineers, or software QA engineers.



ALPHA TESTING



Entry Criteria to Alpha

- ✓ All features are complete/testable (no urgent bugs).
- ✓ High bugs on primary platforms are fixed/verified.
- ✓ 50% of medium bugs on primary platforms are fixed/verified.
- ✓ All features have been tested on primary platforms.
- ✓ Performance has been measured/compared with previous releases (user functions).
- ✓ Usability testing and feedback (ongoing).
- ✓ Alpha sites are ready for installation.

Exit Criteria from Alpha

- ✓ Get responses/feedbacks from customers.
- ✓ Prepare a report of any serious bugs being noticed.
- ✓ Notify bug-fixing issues to developers.
- ✓ Make sure that no more additional features can be included
- ✓ Sign off on Alpha testing

ALPHA TESTING



Advantages of Alpha Testing:

- ✓ Provides better view about the reliability of the software at an early stage
- ✓ Helps simulate real time user behavior and environment.
- ✓ Detect many showstopper or serious errors

Disadvantages of Alpha Testing:

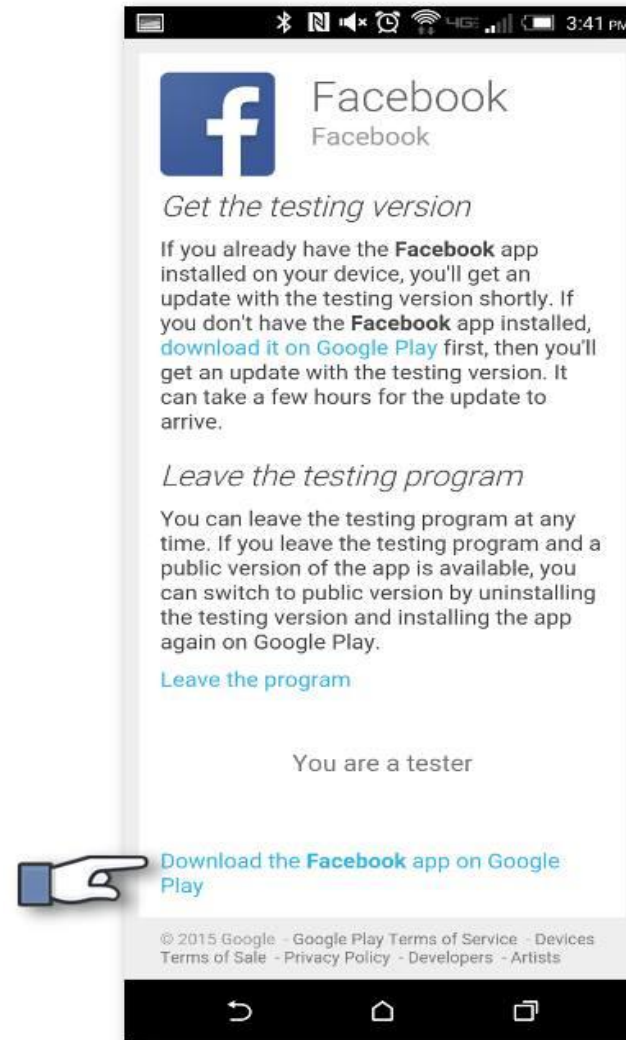
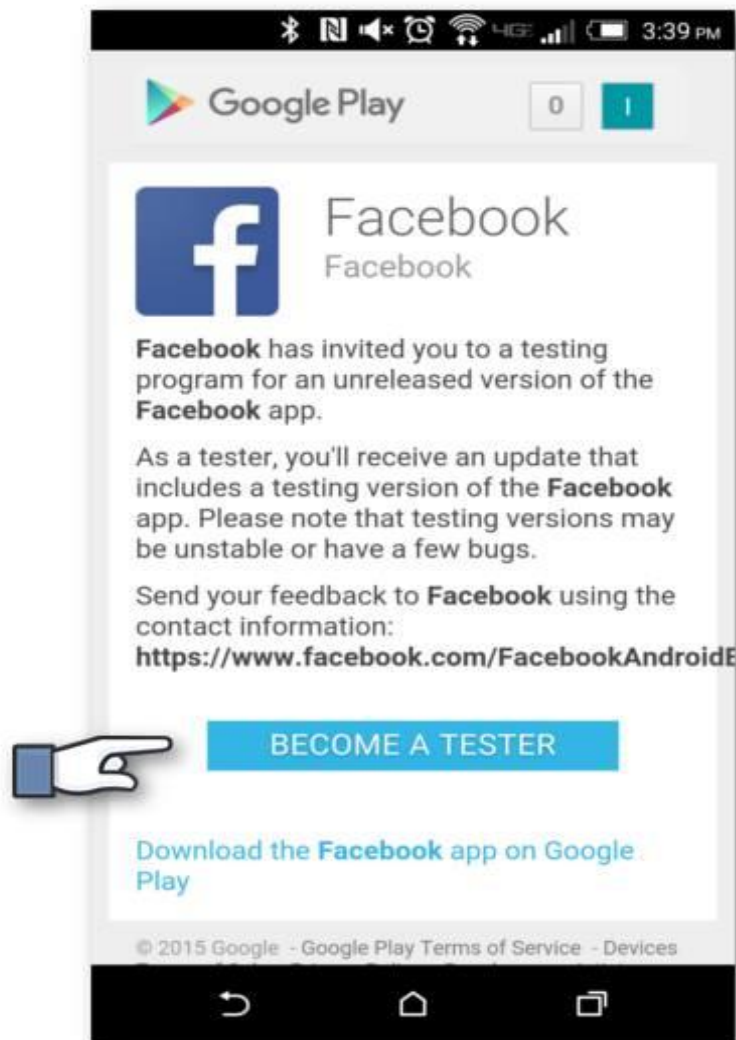
- ✓ In depth functionality of the software cannot be tested as it is still under development stage.

BETA TESTING



- ✓ Beta Testing, also known as “field testing”.
- ✓ Takes place in the customer’s environment and involves some extensive testing by a group of customers who use the system in their environment.
- ✓ Beta Testing is performed by "real users" of the software application in a "real environment" and can be considered as a form of external User Acceptance Testing.
- ✓ Beta version of the software is released to a limited number of end users of the product to obtain feedback on the product quality.
- ✓ Beta testing reduces product failure risks and provides increased quality of the product through customer validation.
- ✓ Beta Test provides a complete overview of the true experience gained by the end users while experiencing the product





Real People, Real Environment, Real Product are the three R's of Beta Testing and the question that arises here in Beta Testing is "Do Customers like the Product?"



Apple Developer

Discover

Design

Develop

Distribute

Support

Account



App Store Connect

Overview

TestFlight

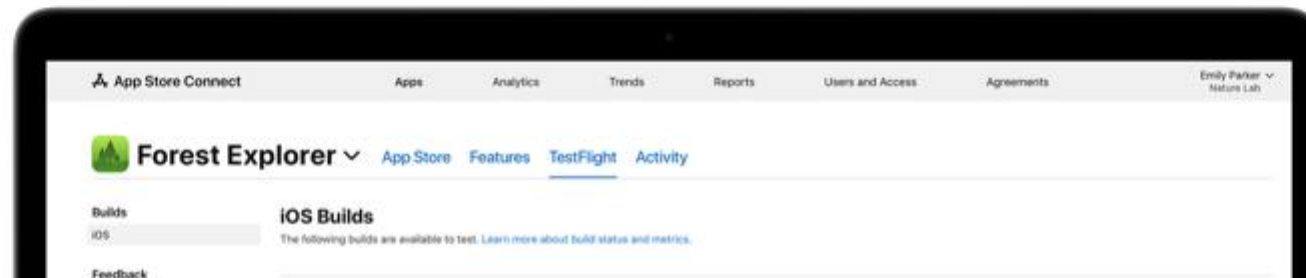
Analytics

API

Sign In

Beta Testing Made Simple with TestFlight

TestFlight makes it easy to invite users to test your apps and app clip experiences and collect valuable feedback before releasing your apps on the App Store. You can invite up to 10,000 testers using just their email address or by sharing a public link.



BETA TESTING



Entrance criteria for Beta Testing:

- ✓ Positive responses from alpha sites.
- ✓ Customer bugs in alpha testing have been addressed.
- ✓ There are no fatal errors which can affect the functionality of the software.
- ✓ Regression testing corresponding to bug fixes has been done.
- ✓ Beta sites are ready for installation.

Exit Criteria for Beta Testing:

- ✓ Get responses/feedbacks from the beta testers.
- ✓ Prepare a report of all serious bugs.
- ✓ Notify bug-fixing issues to developers.

BETA TESTING



Guidelines for Beta Testing:

- ✓ Don't expect to release new builds to beta testers more than once every two weeks.
- ✓ Don't plan a beta with fewer than four releases.
- ✓ If you add a feature, even a small one, during the beta process, the clock goes back to the beginning of eight weeks and you need another 3–4 releases..

BETA TESTING



Advantages of Beta Testing:

- ✓ Reduces product failure risk via customer validation.
- ✓ Beta Testing allows a company to test post-launch infrastructure.
- ✓ Improves product quality via customer feedback
- ✓ Cost effective compared to similar data gathering methods

Disadvantages of Beta Testing:

- ✓ Doesn't allow any control over the testing as it is carried out in real environment and not under the lab environment.
- ✓ Finding the right beta users and maintaining their participation could be a challenge

ALPHA TESTING VS BETA TESTING:



Alpha Testing	Beta Testing
performed by Testers who are usually internal employees of the organization	performed by Clients or End Users who are not employees of the organization
performed at developer's site	performed at client location or end user of the product
Reliability and Security Testing are not performed in-depth Alpha Testing	Reliability, Security, Robustness are checked during Beta Testing
Long execution cycle may be required for Alpha testing	Only few weeks of execution are required for Beta testing

Smoke Testing

SMOKE TESTING



Assume you order an item from **Daraz**.

What will you do after getting the parcel from the Daraz delivery man?

1. check that the parcel is addressed to you
2. and then make sure parcel is intact and not torn.
3. Next, you open the parcel and see the item is what you ordered and also make sure it is new, not old.

Well, you just completed your smoke testing on the parcel.

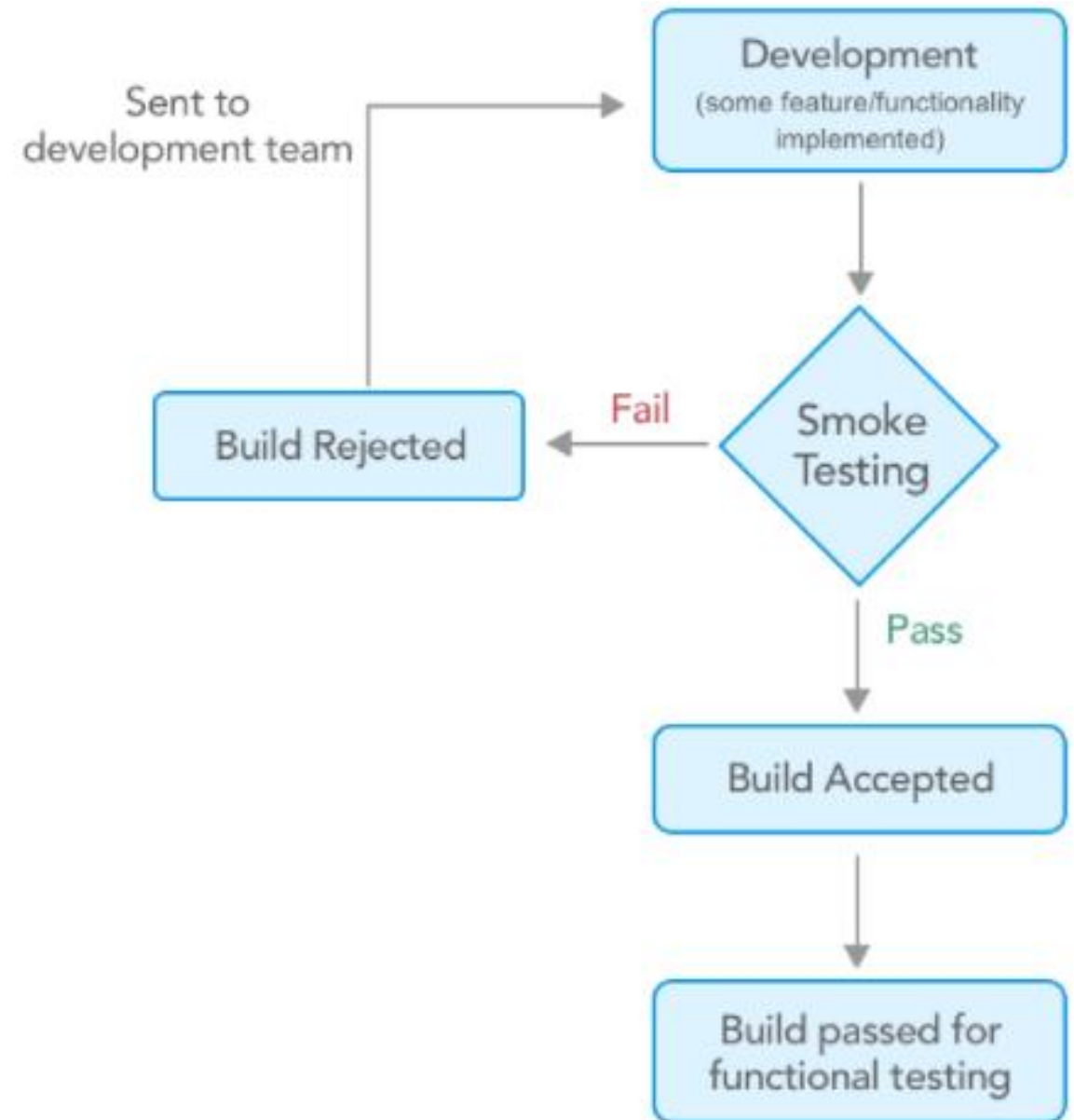
In case of SW testing, Smoke testing is performed on the 'new' build given by developers to QA team to verify if the basic functionalities are working or not.

It is the first test to be done on any new build.

SMOKE TESTING



- ✓ In smoke testing, the test cases chosen cover the most important functionality or component of the system.
- ✓ The objective is not to perform exhaustive testing, but to verify that the critical functionality of the system is working fine.



SMOKE TESTING



Why do we need Smoke Testing

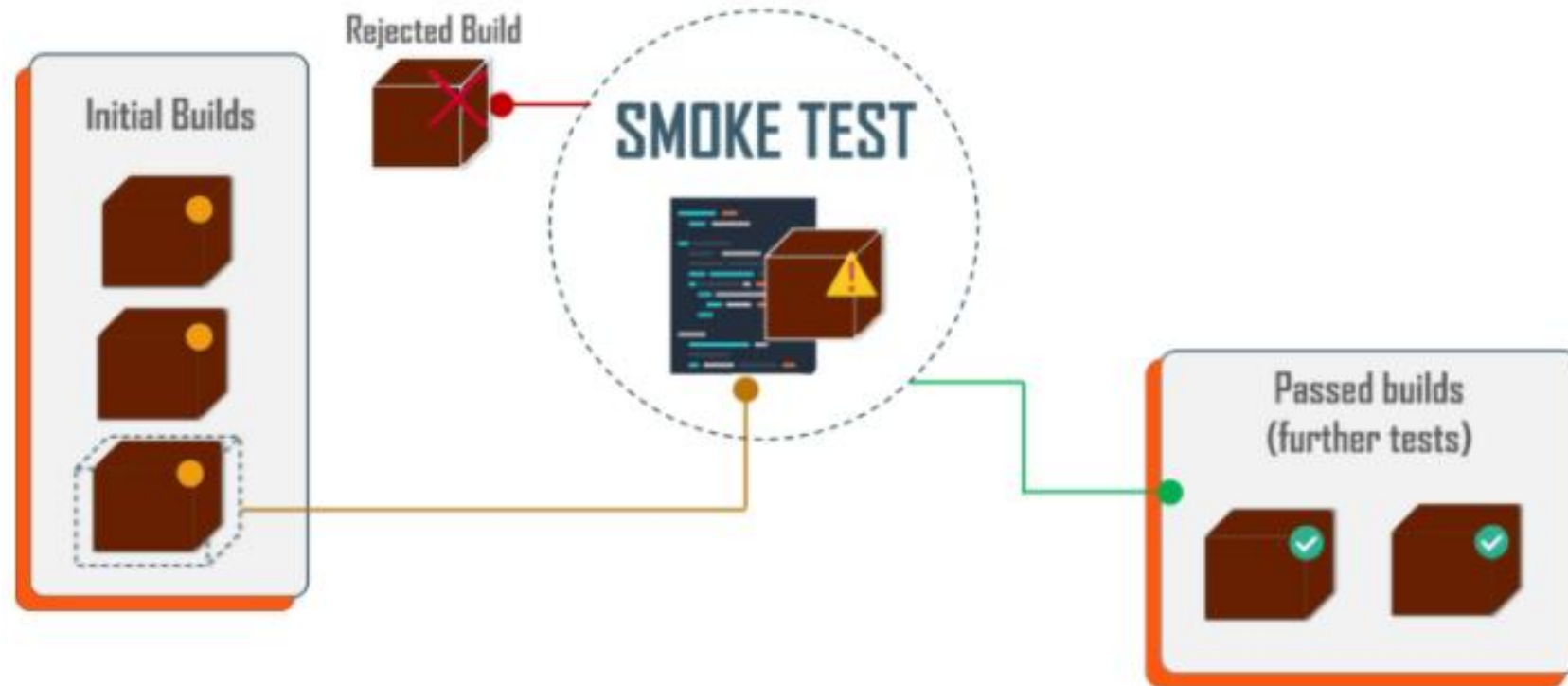
- ✓ Just imagine a situation where you have a testing team consisting of 10 members.
- ✓ all the 10 testers start to test the application and raise the defects for failures they find.
- ✓ Now, at the end of the day, the development team may come back say, sorry this is not the right build or the QA team may stop the testing saying there are too many issues.
- ✓ But again 10 people have already wasted their 8 hours for this which means 80 hours of productivity is lost.
- ✓ This is the reason why we need to have a smoke test done, before jumping into a full-fledged testing cycle.

SMOKE TESTING



When and How Often do We Need Smoke Testing?

- ✓ Smoke Testing normally takes a maximum of 60 minutes and should be done for every new build.
- ✓ Once the product is stable, you can even think about automating the smoke tests.
- ✓ a smoke test is very critical, because it will prevent an unstable or broken build from being pushed into production.



SMOKE TESTING



What are Scenarios that need to be included in a Smoke Test?

- ✓ **Build Verification:** The first and foremost step in a smoke test is to verify the build, the build number, and environment availability. .
- ✓ **Account Creation:** If your application involves a user creation, then you should try to create a new user and check if the system is successfully allowing you to do that
- ✓ **Log in Logout:** If applicable in your SUT (System Under Test), as part of the smoke test you should try to successfully login with old and newly created credentials. also test logout
- ✓ **Business Critical Features:** This is very important. For all the major or business-critical features, we should a simple test to ensure that the most commonly used functionalities are not broken.

SMOKE TESTING



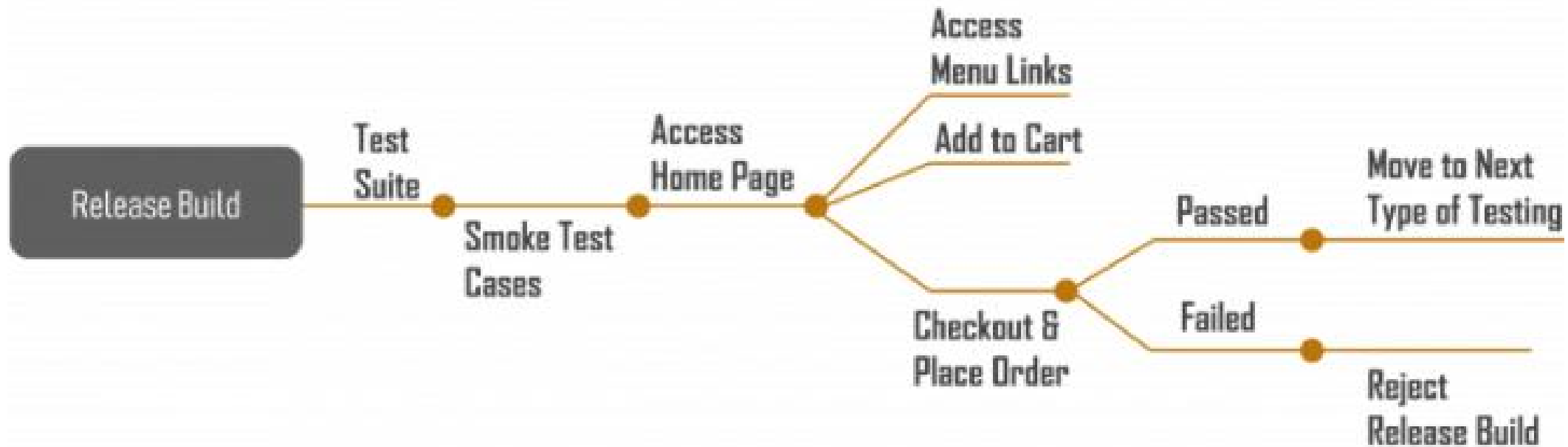
- ✓ **Integration Scenarios:** This is the most important part of a smoke test. The effectiveness of this part depends upon the understanding of the system integrations by the tester. For example, if the tester knows that there is some data that flows from system A to system B, then he must make it a point to check that as part of the smoke test (any 1 value).
- ✓ **Add/Edit/Delete:** Data is always saved in a database. The three basic operations in a database are added the record, edit record and delete a record.
- ✓ **Overall Navigations:** The last part is overall navigation.

SMOKE TESTING



Who will Perform the Smoke Test?

- ✓ Usually QA lead is the one who performs smoke testing. Once the major build of software has been done, it will be tested to find out it's working well or not..
- ✓ The entire QA team sits together and discusses the main features of the software and the smoke test will be done to find out its condition..



SMOKE TESTING



Advantages:

- ✓ It helps to find faults earlier in the product lifecycle.
- ✓ It saves the testers time by avoiding testing an unstable or wrong build
- ✓ It provides confidence to the tester to proceed with testing
- ✓ It helps to find integration issues faster
- ✓ Major severity defects can be found out
- ✓ Detection and rectification will be an easy process
- ✓ Since execution happens quickly, there will be a faster feedback

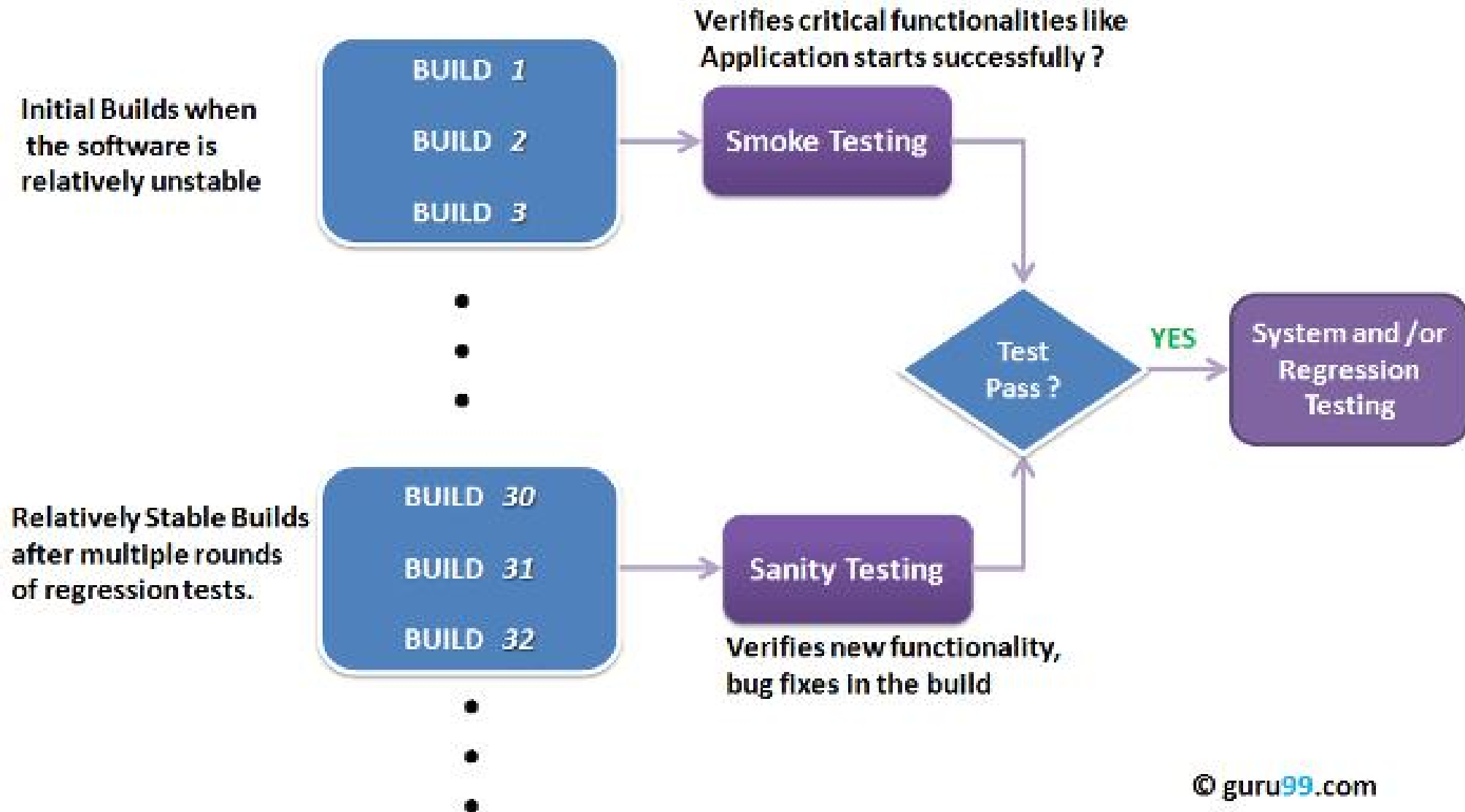
Sanity Testing

SANITY TESTING



- ✓ When a new build is received with minor modifications, instead of running a thorough regression test suite we perform a sanity test.
- ✓ It determines that the modifications have actually fixed the issues and no further issues have been introduced by the fixes.
- ✓ Sanity testing is generally a subset of regression testing and a group of test cases executed that are related to the changes made to the product.

Many testers get confused between sanity testing and smoke testing



Regression Testing

REGRESSION TESTING



Is a type of software testing to confirm that a recent program or code change has not adversely affected existing features.

Is nothing but a **full or partial** selection of **already executed** test cases which are **re-executed** to ensure existing functionalities work fine.

Regression test cases - **carefully selected** to cover as much of the system as possible that can **in any way have been affected** by a maintenance change or any kind of change.

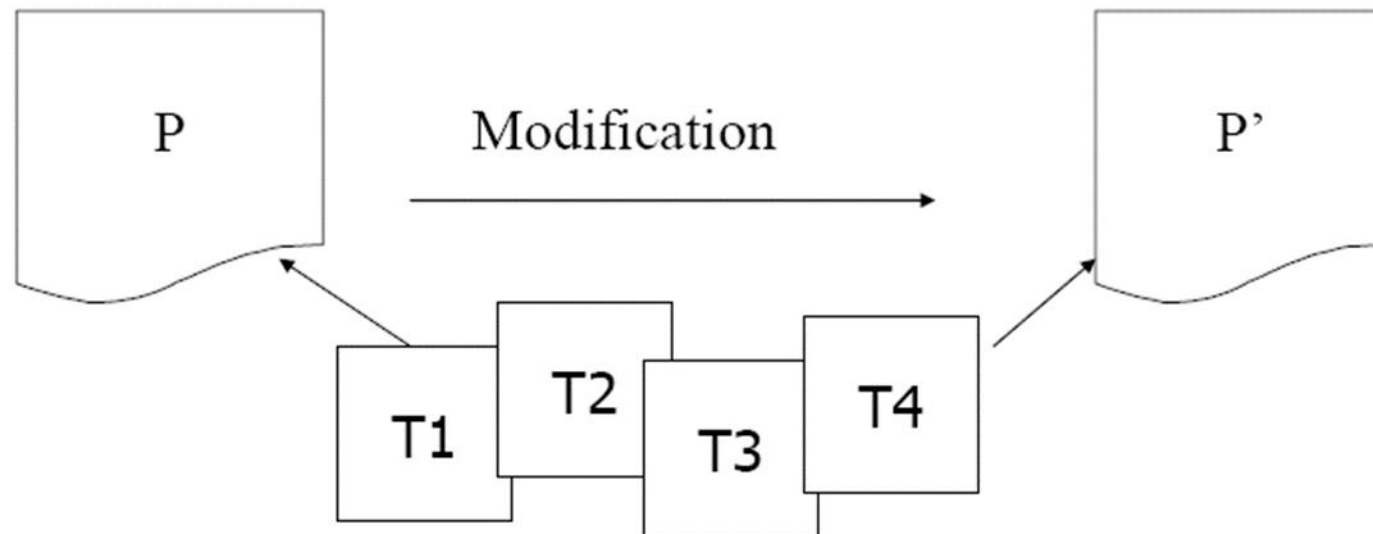
REGRESSION TESTING



Regression testing is the execution of a set of test cases on a program in order to ensure that its **revision** does not produce **unintended faults**, does not "regress" - that is, become less effective than it has been in the past

In a more formal way-

Given program **P**, its modified version **P'**, and a test set **T** that was used to previously test **P**, find a way to utilize **T** to gain sufficient confidence in the correctness of **P'**.



REGRESSION TESTING

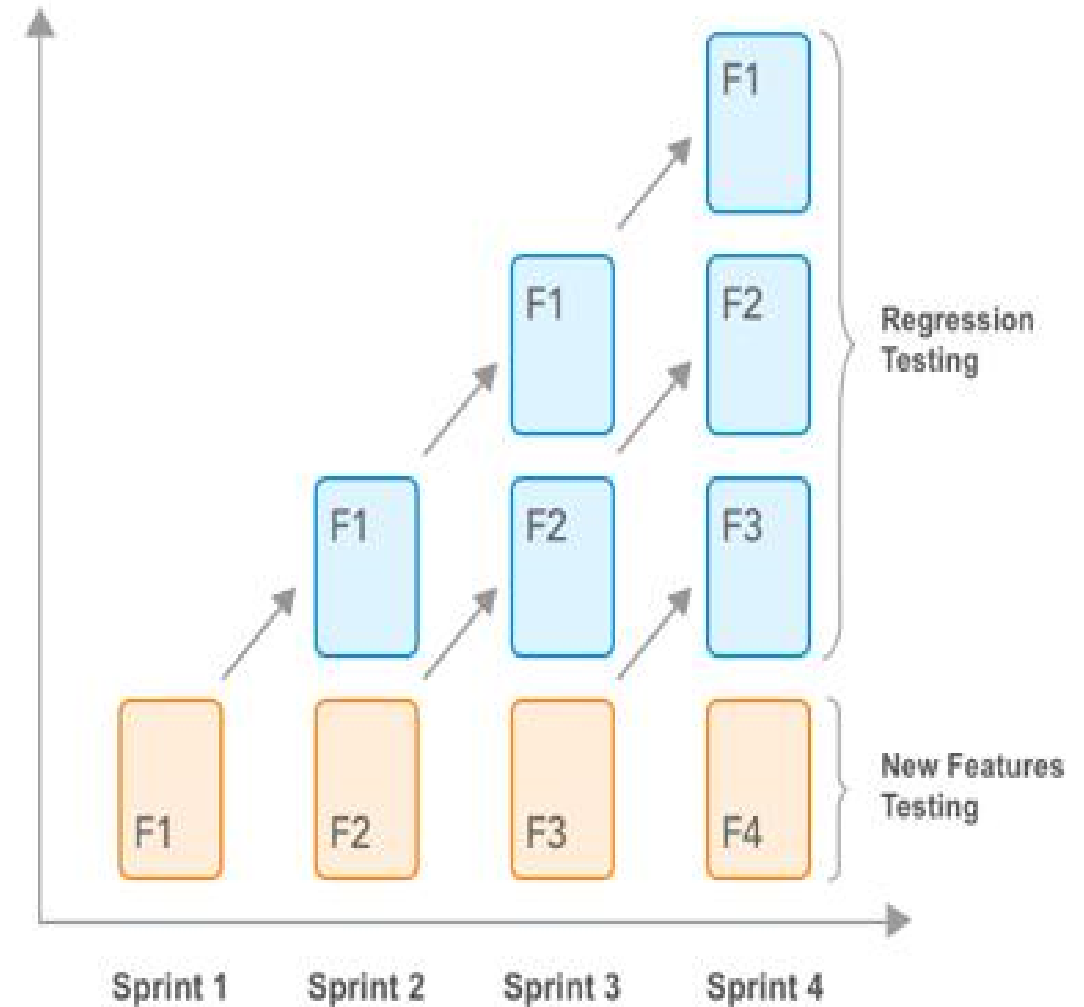


- ✓ Regression testing is crucial during maintenance.
- ✓ It is a good idea to automate regression testing so that all test cases are run after each modification to the software
- ✓ When you find a bug in your program you should write a test case that exhibits the bug.
- ✓ Then using regression testing you can make sure that the old bugs do not reappear

REGRESSION TESTING



- ✓ Regression testing is performed to make sure that a change or addition hasn't broken any of the existing functionality.
- ✓ Its purpose is to find bugs that may have been accidentally introduced into the existing build and to ensure that previously removed bugs continue to stay dead.
- ✓ regression testing is the process of re-testing the modified parts of the software and ensuring that no new errors have been introduced into previously tested source code due to these modifications.



REGRESSION TESTING



Regression testing is the process of re-testing the modified parts of the software and ensuring that no new errors have been introduced into previously tested source code due to these modifications.

Let us check what are the causes to modify a software.....

- (i) Some errors may have been discovered during the actual use of the software.
- (ii) The user may have requested for additional functionality.
- (iii) Software may have to be modified due to change in some external policies and principles.
- (iv) restructuring work may have to be done to improve the efficiency/performance of the SW.
- (v) Software may have to be modified due to change in existing technologies.

REGRESSION TESTING



When and How Often do We Need Regression Testing?

- ✓ We typically think of regression testing as a software maintenance activity

Few cases when we need to perform regression test:

- Addition of new functionalities
- Change requirements
- After bug fix
- Performance issue
- Environment change

Challenges:

- ✓ Time Consuming
- ✓ Expensive
- ✓ Complex

REGRESSION TESTING



Selecting test cases for regression testing

Effective Regression Tests can be done by selecting the following test cases -

- ✓ Test cases which have frequent defects
- ✓ Functionalities which are more visible to the users
- ✓ Test cases which verify core features of the product
- ✓ Test cases which has undergone more and recent changes
- ✓ All Integration Test Cases
- ✓ All Complex Test Cases
- ✓ Boundary value test cases
- ✓ A sample of Successful test cases
- ✓ A sample of Failure test cases

Non Functional Testing

NON-FUNCTIONAL TESTING



The Major Non-Functional Testing techniques include:

- ✓ Performance testing
- ✓ Load testing
- ✓ Stress testing
- ✓ Scalability testing
- ✓ Stability testing
- ✓ Compatibility testing
- ✓ Compliance testing
- ✓ Recovery testing
- ✓ Security testing
- ✓ Usability testing

Performance Testing

PERFORMANCE TESTING



- ✓ is a type of software testing that ensures that the software applications will perform well under their expected workload.
- ✓ The goal of performance testing is not to find bugs but to eliminate performance bottlenecks.
- ✓ Performance testing measures the quality attributes of the system, such as scalability, reliability and resource usage.

PERFORMANCE TESTING



Some of the factors that governs Performance testing:

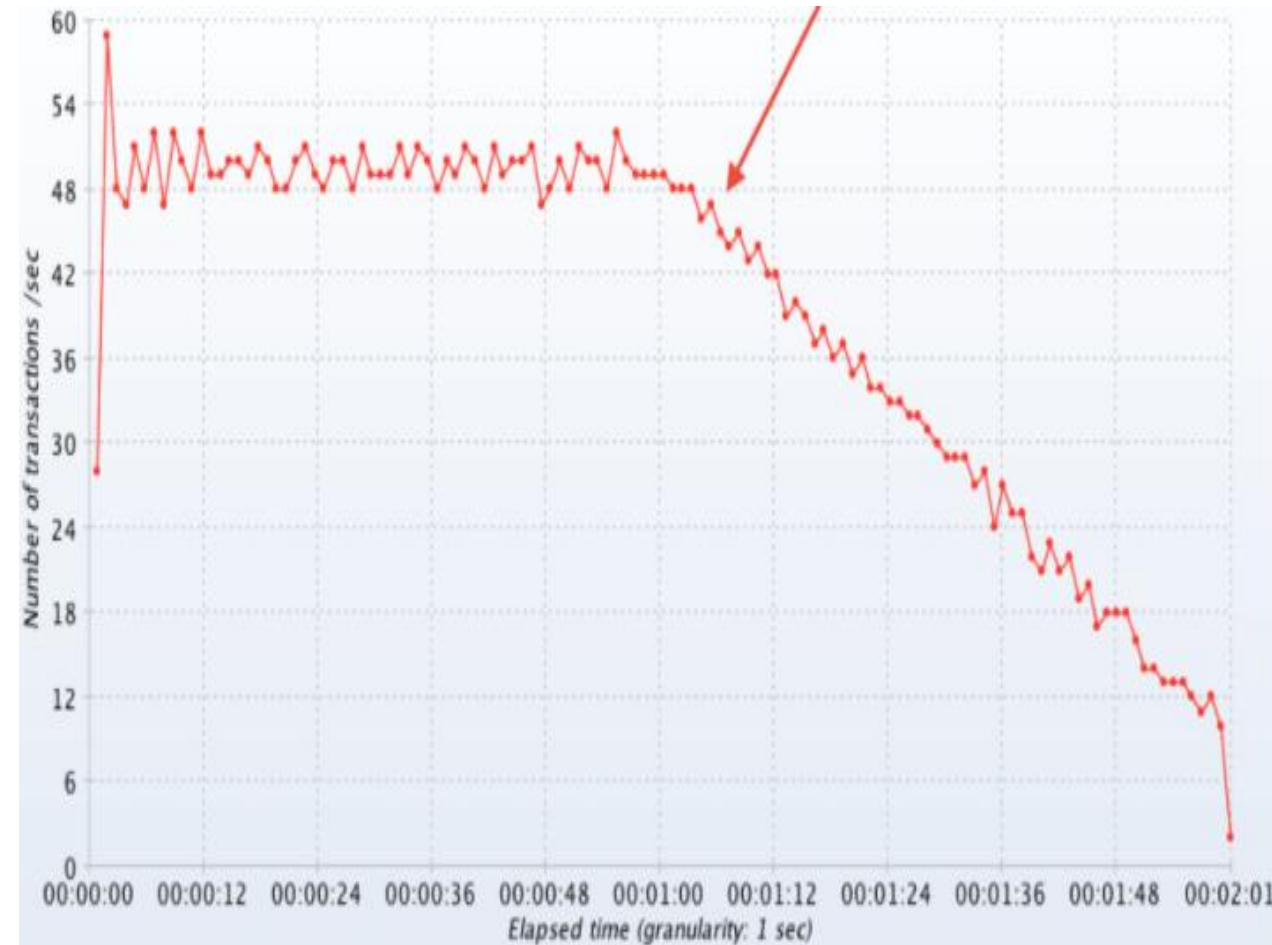
- ✓ Throughput
- ✓ Response Time
- ✓ Tuning
- ✓ Benchmarking

PERFORMANCE TESTING



Throughput:

- ✓ Capability of a product to handle multiple transactions in a give period.
- ✓ Throughput represents the number of requests/business transactions processed by the product in a specified time duration [typically measured as total no. of transaction or requests in a given time or TPS (transaction per second)].
- ✓ It is one of the significant indicator that helps in evaluating the performance of application.

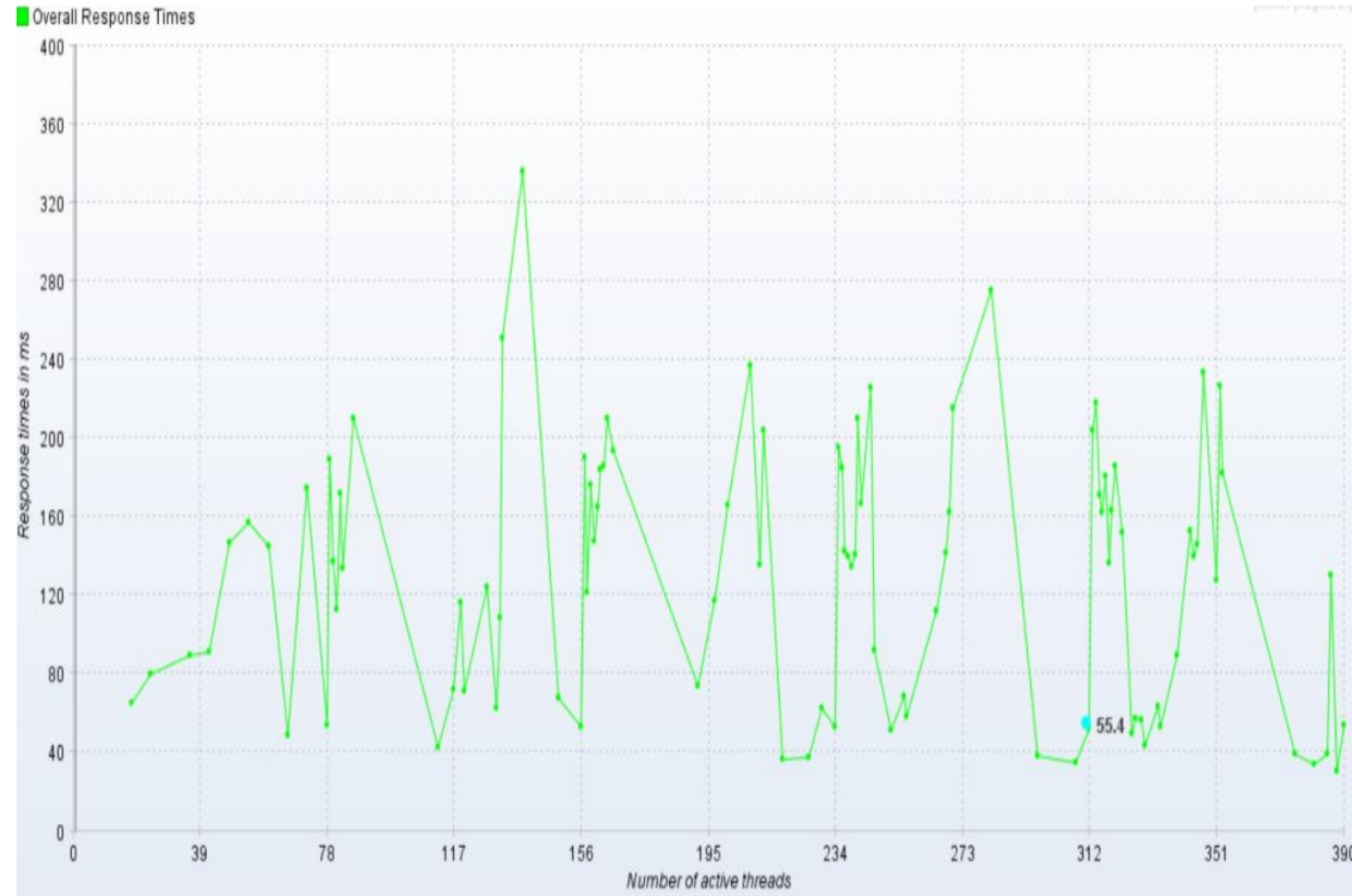


PERFORMANCE TESTING



Response Time:

- ✓ It is equally important to find out how much time each of the transactions took to complete.
- ✓ Response time is defined as the delay between the point of request and the first response from the product.
- ✓ The response time increases proportionally to the user load.

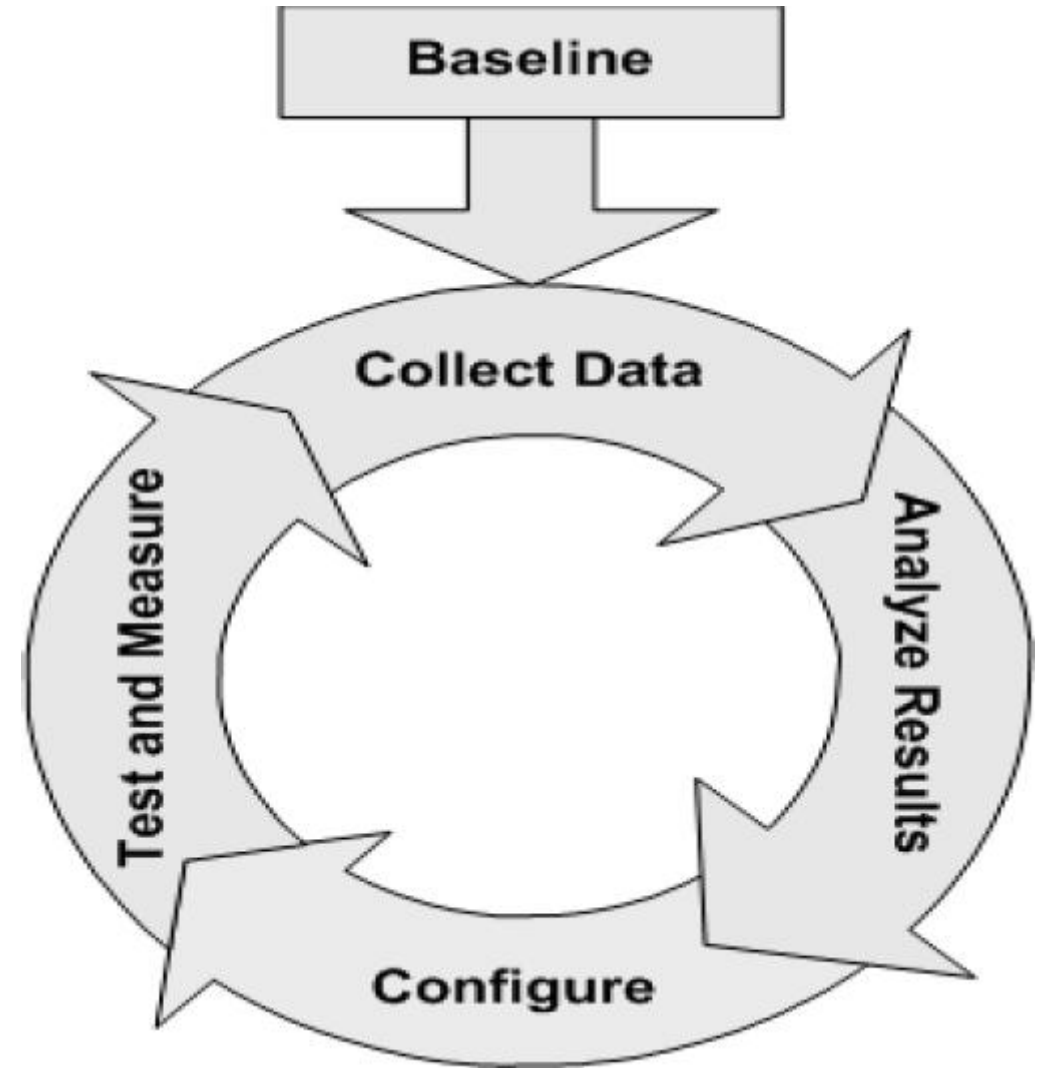


PERFORMANCE TESTING



Tuning:

- ✓ Tuning is the procedure by which product performance is enhanced by setting different values to the parameters of the product, operating system and other components.
- ✓ Tuning improves the product performance without having to touch the source code of the product.

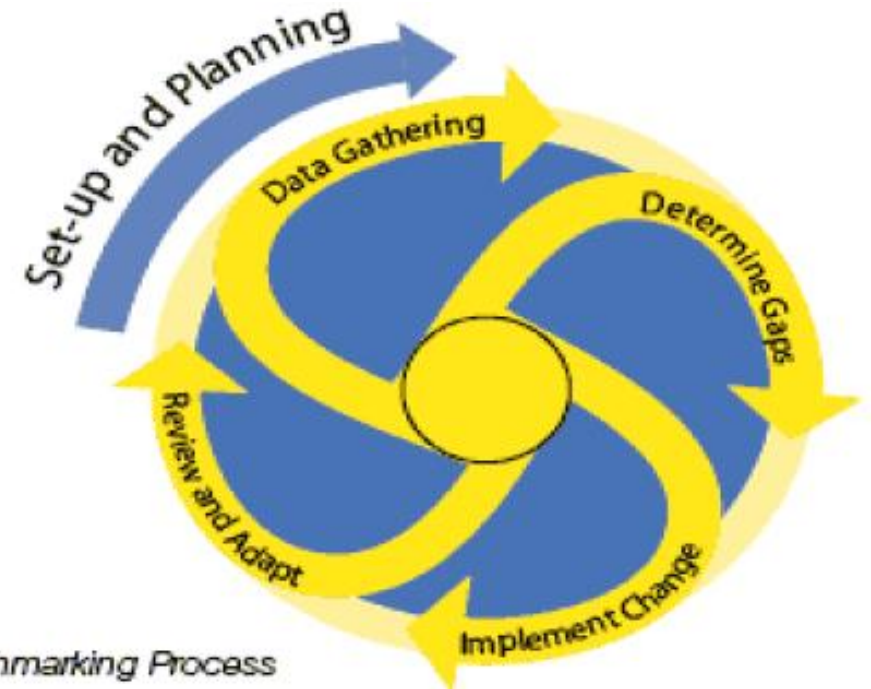


PERFORMANCE TESTING



Benchmarking:

- ✓ A very well-improved performance of a product makes no business sense if that performance does not match up to the competitive products.
- ✓ A careful analysis is needed to chalk out the list of transactions to be compared across products so that an **apple-apple comparison** becomes possible.



The Benchmarking Process

PERFORMANCE TESTING



Finally

- ✓ The testing to evaluate the response time (speed), throughput and utilization of system to execute its required functions in comparison with different versions of the same product or a different competitive product is called Performance Testing.
- ✓ Tuning is performed until the system under test achieves the expected levels of performance.

PERFORMANCE TESTING



What should be tested???

High frequency transactions:

- ✓ The most frequently used transactions have the potential to impact the performance of all of the other transactions if they are not efficient.

Mission Critical transactions:

- ✓ The more important transactions that facilitate the core objectives of the system should be included, as failure under load of these transactions has, by definition, the greatest impact.

Read Transactions:

- ✓ At least one READ ONLY transaction should be included, so that performance of such transactions can be differentiated from other more complex transactions.

Update Transactions:

- ✓ At least one update transaction should be included so that performance of such transactions can be differentiated from other transactions.

PERFORMANCE TESTING



The Process:

1. Identify performance scenarios

Firstly, we will identify the performance scenarios based on these below factors:

- Most commonly scenarios
- Most critical scenarios
- Huge data transaction

2. Plan and design performance test script

In this step, we will install the tools in the Test Engineer Machine and access the test server and then we write some script according to the test scenarios and run the tool.

3. Configure the test environment & distribute the load

After writing the test scripts, we will arrange the testing environment before the execution. And also, manage the tools, other resources and distribute the load according to the "Usage Pattern" or mention the duration and stability.



PERFORMANCE TESTING



The Process:

4. Execute test scripts Once we are done with distributing the load, we will execute, validate, and monitor the test scripts.

5. Result After executing the test scripts, we will get the test result. And check that the result meeting the goal in the given response time or not, and the response time could be maximum, average, and minimum.

6. Analysis result First, we will analyze the test result whether it meets with the response time or not.

7. Identify the Bottleneck After that, we will identify the bottleneck (bug or performance issue). And the bottleneck could occur because of these aspects like the problem in code, hardware issue (hard disk, RAM Processor), network issues, and the software issue (operating system). And after finding the bottleneck, we will perform tuning (fix or adjustment) to resolve this bottleneck.

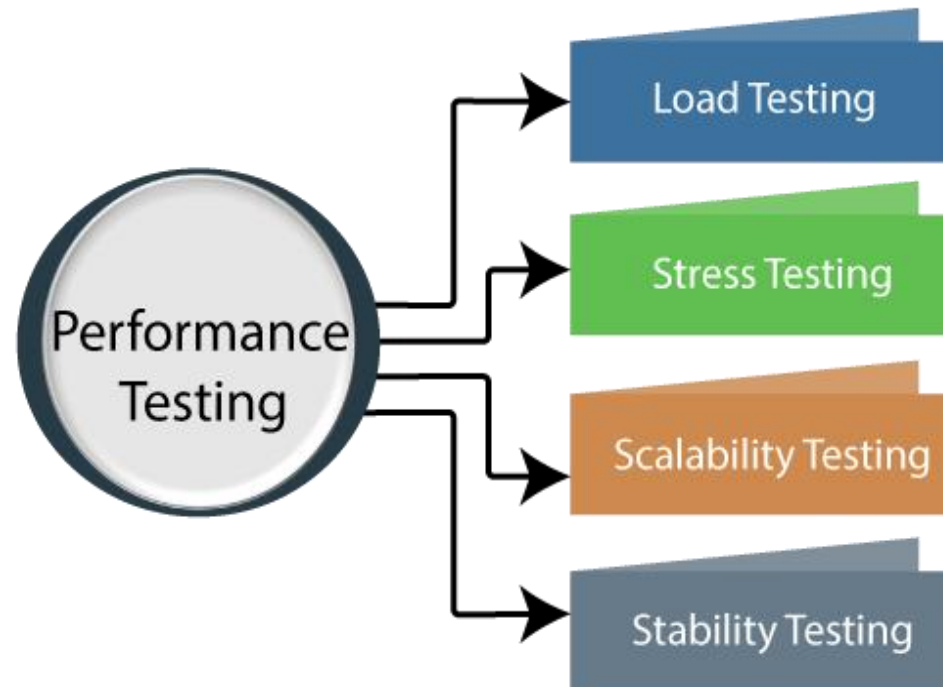
8. Re-run test Once we fix the bottlenecks, re-run the test scripts and checks the result whether it meets the required goal or not.



PERFORMANCE TESTING



Types of Performance Testing:



PERFORMANCE TESTING



Load Testing

- ✓ Load testing is performance testing technique using which the response of the system is measured under various load conditions.
- ✓ The load testing is performed for normal and peak load conditions.
- ✓ is used to check the performance of an application by applying some load which is either less than or equal to the **desired load**.



PERFORMANCE TESTING



Stress Testing:

- ✓ Stress testing a Non-Functional testing technique that is performed as part of performance testing.
- ✓ During stress testing, the system is monitored after subjecting the system to overload to ensure that the system can sustain the stress.
- ✓ That is with stress testing we check the behavior of an application by applying load greater than the desired load.
- ✓ The recovery of the system from such phase (after stress) is very critical as it is highly likely to happen in production environment.

PERFORMANCE TESTING



Reasons for conducting Stress Testing:

- ✓ It allows the test team to monitor system performance during failures/heavy load.
- ✓ To verify if the system has saved the data before crashing or NOT.
- ✓ To verify if the system prints meaning error messages while crashing or did it print some random exceptions.
- ✓ To verify if unexpected failures do not cause security issues.

PERFORMANCE TESTING



Scalability Testing:

- ✓ Checking the performance of an application by increasing or decreasing the load in particular scales (no of a user) is known as scalability testing.
- ✓ Scalability testing is divided into two parts which are as follows:
 - **Upward scalability testing:** It is testing where we increase the number of users on a particular scale until we get a crash point. We will use upward scalability testing to find the maximum capacity of an application.
 - **Downward scalability testing:** The downward scalability testing is used when the load testing is not passed, then start decreasing the no. of users in a particular interval until the goal is achieved. So that it is easy to identify the bottleneck (bug).

PERFORMANCE TESTING



Stability Testing:

- ✓ is a type of non-functional testing that evaluates how well a software application performs over an extended period under normal and stressful conditions.
- ✓ It helps ensure that the system remains stable, responsive, and does not crash due to memory leaks, resource exhaustion, or performance degradation.

Why is Stability Testing Important?

- ✓ Ensures **continuous and reliable** operation of the software.
- ✓ Identifies **performance degradation** over time.
- ✓ Prevents system failures during **long-running operations**.
- ✓ Improves **user experience** by ensuring smooth operation without frequent restarts.

PERFORMANCE TESTING



Tools used for Performance Testing

Open Source

OpenSTA
Diesel Test
TestMaker
Grinder
LoadSim
Jmeter
Rubis

Commercial

LoadRunner
Silk Performer
Qengine
Empirix e-Load

These are well Documented, Should check those with interest...

COMPATIBILITY TESTING

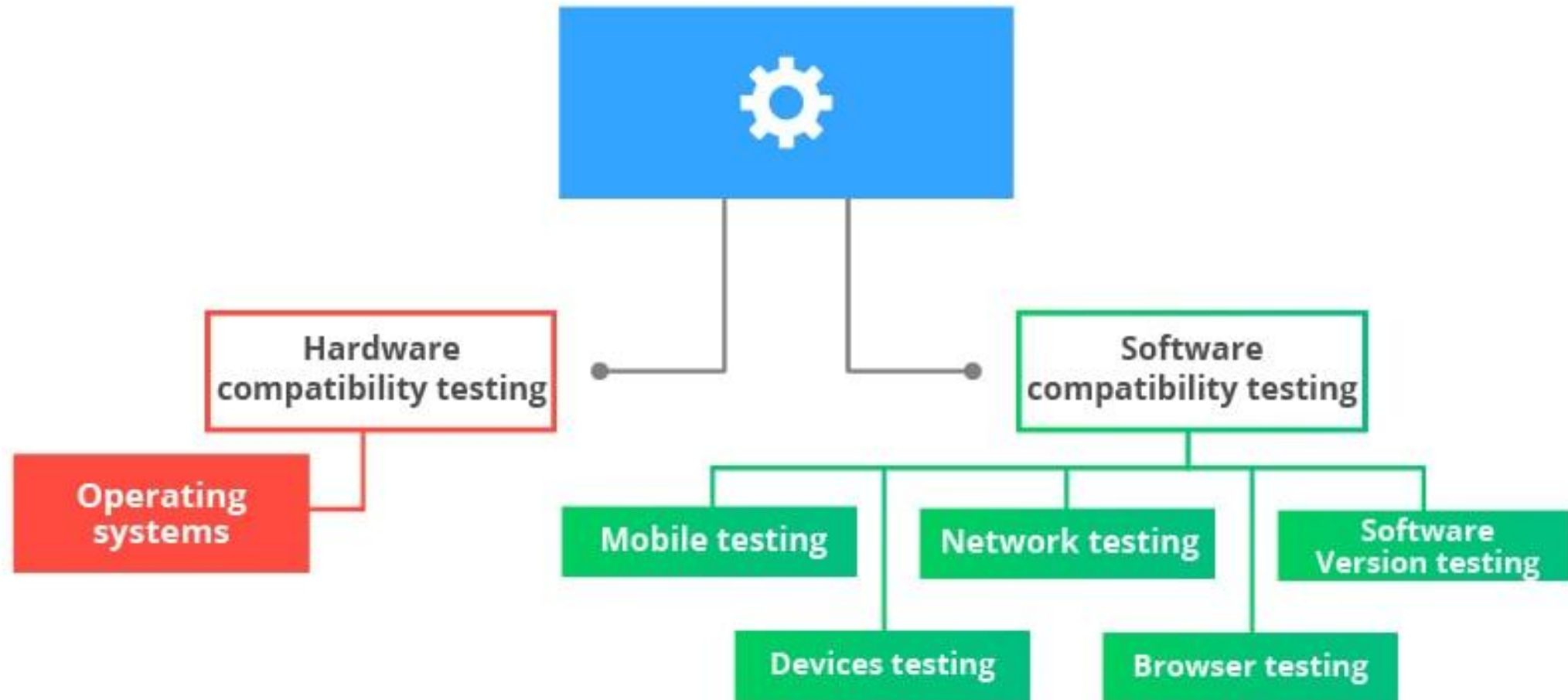


- ✓ Sometimes known as platform testing
- ✓ Compatibility Testing is a type of Non-functional testing
- ✓ Compatibility Testing is a type of Software testing to check whether your software is capable of running on different hardware, operating systems, applications, network environments or Mobile devices.

COMPATIBILITY TESTING



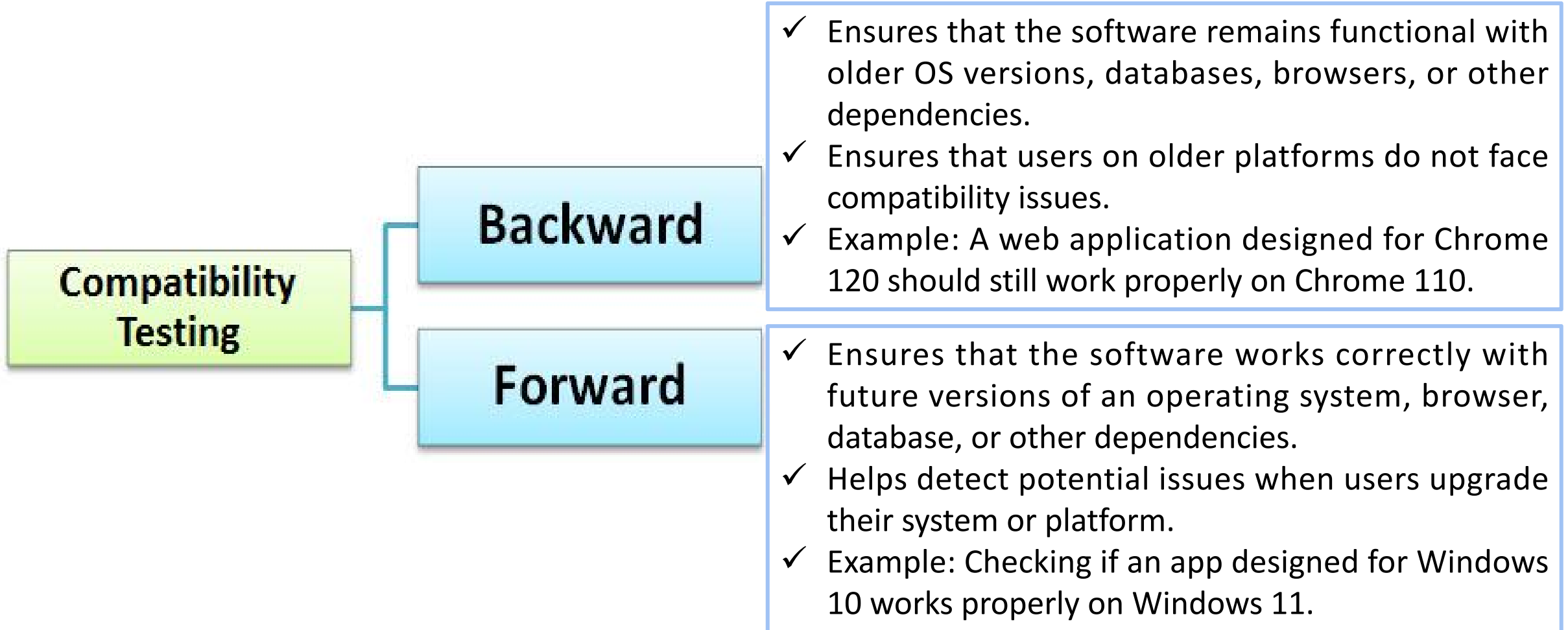
Types of Compatibility Tests



COMPATIBILITY TESTING



There are two types of version checking in **Compatibility Testing** :



COMPLIANCE TESTING



- ✓ also called **Conformance Testing** ensures that a software application meets industry standards, regulations, and legal requirements.
- ✓ It verifies whether the software follows specified guidelines set by regulatory bodies, government agencies, or industry organizations.
- ✓ Types of Compliance Testing
 - ✓ Regulatory Compliance Testing – Ensures adherence to laws and regulations. Example: GDPR (General Data Protection Regulation) compliance for handling user data in the EU.
 - ✓ Security Compliance Testing – Ensures the application meets security standards. Example: Checking if an online banking system follows PCI DSS (Payment Card Industry Data Security Standard) for secure transactions.
 - ✓ Industry-Specific Compliance Testing – Ensures compliance with industry standards. Example: A financial software used by companies to manage accounts must follow SOX (Sarbanes-Oxley Act) to prevent fraud and ensure transparency in financial reporting.
 - ✓ Software Standard Compliance Testing – Ensures adherence to software development standards. Example: Checking if a website follows WCAG (Web Content Accessibility Guidelines) to be accessible for users with disabilities.

RECOVERY TESTING



- ✓ Recovery testing is a type of non-functional testing technique performed in order to determine how quickly the system can recover after it has gone through system crash or hardware failure.
- ✓ Recovery testing is the forced failure of the software to verify if the recovery is successful.

RECOVERY TESTING



Recovery Plan – Steps

1. Determining the feasibility of the recovery process.
2. Verification of the backup facilities.
3. Ensuring proper steps are documented to verify the compatibility of backup facilities.
4. Providing Training within the team.
5. Demonstrating the ability of the organization to recover from all critical failures.
6. Maintaining and updating the recovery plan at regular intervals.

SECURITY TESTING



- ✓ Security testing is a testing technique to determine if an information system protects data and maintains functionality as intended.
- ✓ Process intended to reveal flaws in the security mechanisms of an information system
- ✓ Finding out the potential loopholes & weakness of the system
- ✓ To check whether there is an information leakage

SECURITY TESTING



Aims to verify 6 basic principles as listed below:

- Confidentiality – Is my secret safe...
- Integrity – Is my data tampered..
- Authentication – Who am I? Something you know!! Something you have!
- Authorization – What can I do..
- Availability –Do the information ready for use when expected..
- Non-repudiation - prevent the later denial that an action happened, or a communication that took place

NEXT CLASS



TOPIC NOT DECIDED YET....